Available online at www.sciencedirect.com
ScienceDirect
Comput. Methods Appl. Mech. Engrg. 401 (2022) 115616
Computer methods in applied mechanics and engineering
www.elsevier.com/locate/cma

A mixed formulation for physics-informed neural networks as a potential solver for engineering problems in heterogeneous domains: Comparison with finite element method

Shahed Rezaei[a,*], Ali Harandi[b], Ahmad Moeineddin[b], Bai-Xiang Xu[a], Stefanie Reese[b]

[a] *Mechanics of Functional Materials Division, Institute of Materials Science, Technical University of Darmstadt, Otto-Berndt-Str. 3, D-64287 Darmstadt, Germany*

[b] *Institute of Applied Mechanics, RWTH Aachen University, Mies-van-der-Rohe-Str. 1, D-52074 Aachen, Germany*

Received 15 June 2022; received in revised form 25 August 2022; accepted 25 August 2022
Available online 20 September 2022

Abstract

Physics informed neural networks (PINNs) are capable of finding the solution for a given boundary value problem. Here, the training of the network is equivalent to the minimization of a loss function that includes the governing (partial differential) equations (PDE) as well as initial and boundary conditions. We employ several ideas from the finite element method (FEM) to enhance the performance of existing PINNs in engineering problems. The main contribution of the current work is to promote using the spatial gradient of the primary variable as an output from separated neural networks. Later on, the strong form (given governing equation) which has a higher order of derivatives is applied to the spatial gradients of the primary variable as the physical constraint. In addition, the so-called energy form of the problem (which can be obtained from the weak form) is applied to the primary variable as an additional constraint for training. The proposed approach only required up to first-order derivatives to construct the physical loss functions. We discuss why this point is beneficial through various comparisons between different models. The mixed formulation-based PINNs and FE methods share some similarities. While the former minimizes the PDE and its energy form at given collocation points utilizing a complex nonlinear interpolation through a neural network, the latter does the same at element nodes with the help of shape functions. We focus on heterogeneous solids and check the performance of the proposed PINN model against the solution from FEM on two prototype problems: elasticity and the Poisson equation (steady-state diffusion problem). It is concluded that by properly designing the network architecture in PINN, the deep learning model has the potential to solve the unknowns in a heterogeneous domain without any available initial data from other sources. Finally, discussions are provided on the combination of PINN and data from physical FE simulations for a fast and accurate design of composite materials in future developments.

Keywords: Physical informed neural networks; Unsupervised learning; Heterogeneous solids

1. Introduction

Deep learning (DL) methods as a branch of machine learning (ML) algorithms and artificial intelligence have gained attention in various fields of science. The ability of these models to realize the patterns in the available

* Corresponding author.
E-mail addresses: shahed.rezaei@tu-darmstadt.de (S. Rezaei), ali.harandi@ifam.rwth-aachen.de (A. Harandi).

https://doi.org/10.1016/j.cma.2022.115616
0045-7825/

data makes them very attractive candidates for fast and reliable predictions. These methods also find their way to many engineering applications, especially in material mechanics and design [1]. One significant advantage of DL algorithms is the huge speedup that we can gain upon successful training. The latter point is particularly important when it comes to multiscaling analysis where we would like to have the material properties at the upper scale which are under the influence of the features at the lower scale. Transforming this information is a tedious task; therefore, DL can play a huge role in reducing the complexity (e.g. see Peng et al. [2], Fernández et al. [3], Yin et al. [4]).

Here, we would like to examine the potential of the DL method in particular, for the field of computational engineering, where we are dealing with a complex and perhaps coupled system of governing equations. Moreover, it is vital to clarify different ways based on which one can apply machine learning methods. The first main-stream goes towards application of DL to find the solution using supervised learning based on available data. Clearly, in these contributions, one requires to perform enough off-line calculations or measurements to provide the required data for the network training. The performance of the models in this case theoretically should be improved with increasing training data. Hsu et al. [5] presented an ML approach to predict fracture processes based on data from the molecular simulation and the image-processing approaches. Mianroodi et al. [6] applied a U-Net approach to predict stress fields in heterogeneous non-linear elastoplastic material systems. Bhaduri et al. [7] considered a fiber-reinforced matrix composite material system and used a convolutional neural network (CNN/U-Net architecture) to evaluate stress fields. See also [8,9] for finding structure–property relation utilizing CNN approaches. The application of DL for multiscaling is well-established and has shown its performance in various fields. See for example [2,9–12]. The idea here is to gather data by performing simulations utilizing well-established solvers (such as finite elements, molecular dynamics, etc.). Based on the obtained data, a proper NN is trained to transfer the information to the upper scale [3,8]. In other words, the NN relates the inputs at the lower scale to the required outputs at the upper scale. These NNs require enough data for an accurate prediction, and they do not necessarily satisfy physical constraints governing the given problem. Some recent developments tend to enhance the architecture and design of the NNs based on the application-dependent physical constraints. See for example [13–15]. Further improvements and studies in this direction are essential. Therefore, utilizing physics-informed NN can be very advantageous for this purpose. Note that in this category, the initial and labeled data comes from other resources (either other numerical simulations or experimental measurements) and we still need developments in those fields to gather enough data for training. In other words, here the classical solvers and measures go hand in hand with the machine learning approaches.

Another mainstream is to employ DL methods to find the solution for a boundary value problem (BVP) without any labeled data and solely based on governing equations/physics (unsupervised learning). This is done by introducing the governing equations together with initial and boundary conditions as additional terms in the loss function of the neural networks. The latter is also known in the community as physics informed neural networks (PINNs) [16]. Here, the performance of the DL does not improve with the number of BVPs, just like a finite element solver. Early investigation in this direction seem promising [16–19]. The main advantage of these approaches is that the physical laws are integrated into the network's loss function. As a result, upon proper training, one is confident that the network's outcome respects physical laws even for unseen data and blind predictions in the domain of training. In other words, one deals with a constraint minimization problem which results in the solution of the given boundary value problem [16]. This approach is explored in earlier times to some extent. Lee and Kang [20] employed neural minimization to solve the finite difference PDEs. Psichogios and Ungar [21] used the mixed formulation based neural network which utilized the available prior knowledge about the process and showed significantly less requirement of the ground truth data. Lagaris et al. [22] utilized artificial neural networks for solving boundary value problems, and they compared their results to the ones in which FEM is used. The PINN is further developed, and discussed in detail for various types of PDEs in different branches of physics and engineering (Raissi et al. [16]). It is shown how easily one has access to derivatives of the output variable with respect to the input parameters thanks to automatic differentiation (AD). This idea was picked up by many authors and applied to various fields from fluid mechanics to solid mechanics. For a review of the approach and some recent developments, see [1,23,24]. PINNs is also applied to arbitrary complex domains. Berg and Nyström [25] used deep feedforward artificial neural networks to approximate solutions to PDEs in complex geometries. Blechschmidt and Ernst [26] introduced several ways for solving PDEs via ML-based algorithms and compared the formulation of PINNs against other available approaches. Zobeiry and Humfeld [27] developed a PINN to solve conductive heat transfer equation as well as convective heat transfer in manufacturing applications.

DL methods are also applied in the context of solid mechanics. Samaniego et al. [28] explored deep neural networks for approximation of the solution to PDEs and analyzed the energetic format of the PDE. The energy

of the given system is introduced as a natural loss function for a machine learning method, see also [29]. Guo et al. [18] presented a deep collocation method (DCM) in non-homogeneous media which utilizes a PINN with material transfer learning. The authors studied potential problems such as heat transfer and electric conduction. See also [30] where authors presented applications of PINNs to heat transfer problems. Haghighat et al. [31] applied PINNs to solid mechanics. The authors investigated linear elastic problems and extended the formulation for elasto-plastic behavior. See also [32]. Zhang et al. [33] presented a framework based on PINNs for identifying unknown geometric and material parameters. The authors compared their predictions for materials with internal voids/inclusions for the case of linear elasticity, hyperelasticity, and plasticity. Abueidda et al. [34] combined deep energy method formulation and DL to solve the governing PDEs of the deformation in hyperelastic and viscoelastic materials. The authors show the capability of the method to accurately capture the three-dimensional mechanical response without requiring any time-consuming training data.

Many of the mentioned contributions are applied to homogeneous material systems. According to our investigation, they are not very suitable for heterogeneous systems, where we have a sharp contrast between different phases. Therefore, further refinements and extensions are required. Rao et al. [35] presented a PINN with mixed-variable output to model elastodynamics problems without resorting to labeled data, where the initial and boundary conditions are hardly imposed. In other words, the network design automatically satisfies the particular given BCs. The authors considered both the displacements and stress components as the network's outputs. Interestingly, this idea was also investigated before in the hybrid FE analysis. As a result, the accuracy and trainability of the network improved significantly. Yadav et al. [36] solved a series of problems in solid mechanics using a domain-distributed version of distributed PINN and demonstrated that the approach is capable of addressing volumetric locking or capturing discontinuities. Henkes et al. [19] addressed non-homogeneous materials which can trigger the nonlinearities in the stress and displacement fields. They have also utilized domain decomposition to get more accurate solutions. See also investigations by [17,37–39]. It is worth mentioning that the methodologies above can be categorized as mesh-free approaches in numerical modeling. As we will explain through this work, collocation points at which the loss functions are minimized, can be interpreted similarly to nodes in finite element methods.

Similar to other numerical solvers, PINNs have the potential to be utilized for addressing multiphysics problems where we have a multi-objective optimization problem to satisfy several coupled governing equations. He et al. [40] employed PINNs in subsurface transport problems to minimize the governing equations of the problem in addition to loss functions based on the measurement data. Nguyen et al. [41] utilized PINNs and the domain decomposition method to solve coupled problems for some benchmarks test, and they showed the capability of the model when it comes to a limited amount of data. Patel et al. [42] introduced a method for shock hydrodynamics that can satisfy hyperbolicity. Goswami et al. [43] developed a network for the phase-field model of the fracture (see also Goswami et al. [44]). Amini et al. [45] investigated the application of PINNs to the solution of problems involving thermo-hydro-mechanical processes.

This work will discuss a simple yet effective extension of the PINN approach to directly solve a given PDE and boundary condition in a heterogeneous domain (see Fig. 1). In Section 2, the formulation of the problem for linear elasticity and steady-state diffusion problem is described. For the diffusion problem, we mainly focused on the heat transfer but it can be directly interpreted as an electrostatic, electric conduction, or magnetostatic problem. The derivation of the energy form from the PDE is also shortly reviewed for each of the described physics to show its applicability to other similar problems. In Section 3, the architecture and details of the feed-forward neural network are described where we summarize all the necessary loss terms. This work is implemented using the SciANN package Haghighat and Juanes [46] but the methodology can easily be transferred to other platforms for similar studies. Finally, we provide a conclusion and future ideas on how this work can be extended for fast and accurate prediction of material behavior in multi-scaling scenarios.

2. Formulation of the problem

2.1. Linear elasticity in heterogeneous solids

In the context of linear elasticity, the formulation of the mechanical problem is described by three main equations: kinematics, constitutive relation (Hooke's law), and mechanical equilibrium. The kinematic relation defines the strain tensor $\boldsymbol{\varepsilon}$ in terms of the deformation vector $\boldsymbol{u}$ and reads

$$\boldsymbol{\varepsilon} = \text{sym}(\text{grad}(\boldsymbol{u})) = \frac{1}{2}\left(\nabla \boldsymbol{u} + \nabla \boldsymbol{u}^T\right). \tag{1}$$

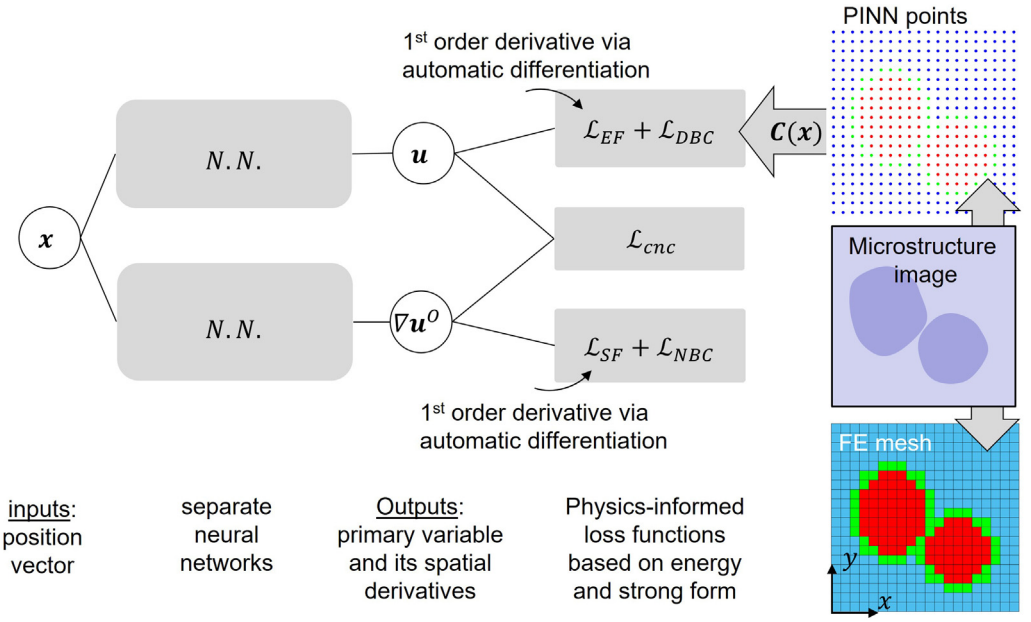

Fig. 1. Proposed network architecture for solving a given boundary value problem. The vector $\boldsymbol{u}$ stands for any unknown primary variable (solution) that we are looking for. The idea is to combine the weak and strong forms of the problem simultaneously using separate networks for the primary variable and its spatial gradients. The networks are finally connected by means of an additional loss function.

Employing the fourth order elasticity tensor $\mathbb{C}$, the second order stress tensor reads

$$\boldsymbol{\sigma} = \mathbb{C}(x, y)\, \boldsymbol{\varepsilon}. \tag{2}$$

Note that we are, in particular, interested in a heterogeneous domain where material properties may vary throughout the microstructure. The latter explains the dependency of the stiffness tensor on the coordinates of collocation points. The mechanical equilibrium in the absence of body force as well as the Dirichlet and Neumann boundary conditions are written as

$$\operatorname{div}(\boldsymbol{\sigma}) = \mathbf{0} \quad \text{in} \quad \Omega \tag{3}$$

$$\boldsymbol{u} = \bar{\boldsymbol{u}} \quad \text{on} \quad \Gamma_D \tag{4}$$

$$\boldsymbol{\sigma} \cdot \boldsymbol{n} = \boldsymbol{t} = \bar{\boldsymbol{t}} \quad \text{on} \quad \Gamma_N \tag{5}$$

In the above relations, Ω and Γ denote the material points in the body and on the boundary area, respectively. For convenience, the second-order strain tensor in Eq. (2) is usually written in the Voigt notation as

$$\hat{\boldsymbol{\sigma}} = \hat{C}(x, y)\, \hat{\boldsymbol{\varepsilon}}. \tag{6}$$

In Eq. (6), $\hat{\bullet}$ denotes the tensor $(\bullet)$ in the Voigt notation. Considering the plane strain assumption in a two dimensional setting, we write the position-dependent elasticity tensor $\hat{\mathcal{C}}$ as (see also Fig. 1):

$$\hat{C}(x, y) = \frac{E(x, y)}{(1 - 2\nu(x, y))(1 + \nu(x, y))} \begin{bmatrix} 1 - \nu(x, y) & \nu(x, y) & 0 \\ \nu(x, y) & 1 - \nu(x, y) & 0 \\ 0 & 0 & \dfrac{1 - 2\nu(x, y)}{2} \end{bmatrix}. \tag{7}$$

Here, Young's Modulus E and Poisson's ratio ν represent the elastic constants for the material. We assume that the material behavior remains isotropic in each phase. Nevertheless, one can also focus on fully anisotropic material by defining $\hat{\mathcal{C}}$ differently and the formulation stays the same. To obtain the Galerkin weak form (WF) of the problem we first multiply Eq. (3) by a test function $\delta\boldsymbol{u}$, which satisfies the Dirichlet boundary conditions of the given boundary

value problem. By applying integration by parts and utilizing Gauss theorem one obtains

$$\int_{\Omega} \delta\hat{\boldsymbol{\varepsilon}}^T \, \hat{C}(x, y) \, \hat{\boldsymbol{\varepsilon}} \; dV \; - \int_{\Gamma_N} \delta \boldsymbol{u}^T \, \bar{\boldsymbol{t}} \; dA = 0. \tag{8}$$

The WF in Eq. (8) leads to another representation which is based on the balance between the mechanical internal energy E^M_{int} and mechanical external energy E^M_{ext}. In other words, we can also represent the problem based on the minimization of the total potential energy in which we have the contribution of internal and external forces (see also [28]).

$$\text{Minimize: } \underbrace{\int_{\Omega} \frac{1}{2} \hat{\boldsymbol{\varepsilon}}^T \, \hat{C}(x, y) \, \hat{\boldsymbol{\varepsilon}} \; dV}_{E^M_{\text{int}}} - \underbrace{\int_{\Gamma_N} \boldsymbol{u}^T \, \bar{\boldsymbol{t}} \; dA}_{E^M_{\text{ext}}}, \tag{9}$$

$$\text{Subjected to: BCs in Eq. (4) and Eq. (5).} \tag{10}$$

The expressions in Eqs. (9) and (10) will be added directly to the loss function of the neural network for the primary variable $\boldsymbol{u}$ (see also [28]).

2.2. Diffusion problem in heterogeneous solids

We consider another prototype problem: the steady-state diffusion problem in a 2-D setting. The equations are described for the problem of heat transfer. Note that the same derivation can be applied to study other similar fields such as electrostatic, electric conduction, or magnetostatic problem [18]. The heat flux $\boldsymbol{q}$ is defined according to Fourier's law:

$$\boldsymbol{q} = -k(x, y) \, \nabla T. \tag{11}$$

Here $k(x, y)$ is the position-dependent heat conductivity coefficient. The governing equation for this problem is based on the heat balance in the absence of heat source and reads

$$\text{div}(\boldsymbol{q}) = 0 \qquad \text{in } \Omega, \tag{12}$$

$$T = \bar{T} \qquad \text{on } \Gamma_D, \tag{13}$$

$$\boldsymbol{q} \cdot \boldsymbol{n} = q_n = \bar{q} \qquad \text{on } \Gamma_N. \tag{14}$$

In the above relations, the Dirichlet and Neumann boundary conditions are introduced in Eqs. (13) and (14), respectively. Similar to the procedure in the mechanical problem, by introducing δT as a test function, the weak form of the steady-state diffusion problem reads

$$\int_{\Omega} k(x, y) \, \nabla^T T \, \delta(\nabla T) \; dV \; + \int_{\Gamma_N} \bar{q} \; \delta T \; dA \; = \; 0. \tag{15}$$

Analogously to the previous section, the weak form of the thermal problem can be interpreted as the variation of the energy and allows us to introduce an equivalent balance between the thermal internal energy E^T_{int} and the thermal external energy E^T_{ext}. Therefore, we reformulate the problem as an energy minimization:

$$\text{Minimize: } \underbrace{\int_{\Omega} \frac{1}{2} k(x, y) \, \nabla^T T \, \nabla T \; dV}_{E^T_{\text{int}}} + \underbrace{\int_{\Gamma_N} \bar{q} \; T \; dA}_{E^T_{\text{ext}}}, \tag{16}$$

$$\text{Subjected to: BCs in Eq. (13) and Eq. (14).} \tag{17}$$

In the expression for the loss function, the above term is evaluated for the primary variable T.

In the current study, we study two different set of geometries according to Fig. 2. They are selected to cover some important cases which happen in different engineering problems. In addition to phase changes (geometry 1), sharp corners between different phases are introduced in geometry 2. Material properties are varied for different phases. The mechanical and thermal problem described in Section 3.1 and Section 3.5, respectively. For these two problems, we change the Young's modulus and thermal conductivity of the involved material according the relation

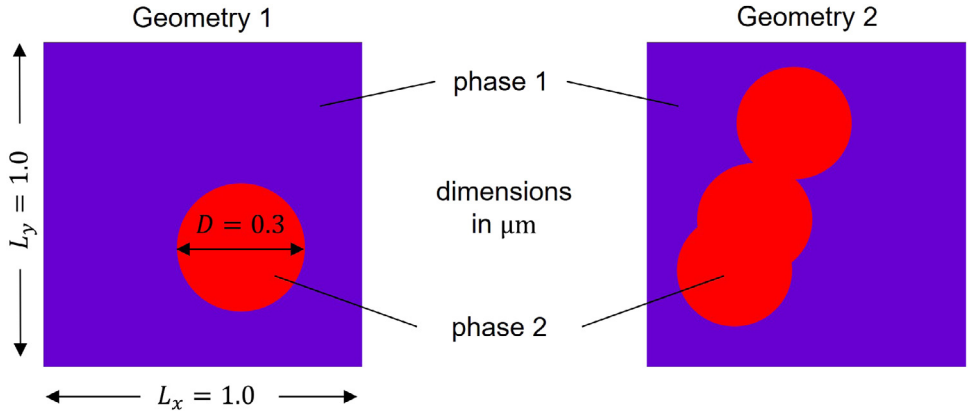

Fig. 2. Selected geometries for studies on mechanical and steady-state diffusion problem.

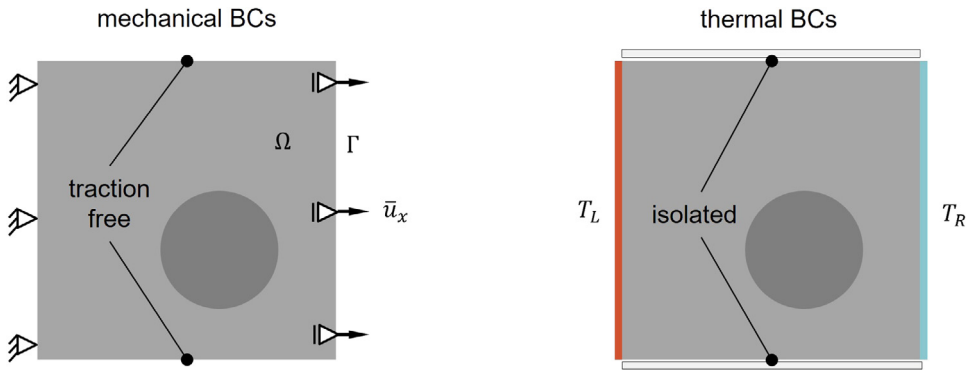

Fig. 3. Boundary conditions for mechanical and thermal problems.

$E_{\text{inc}}/E_{\text{mat}} = k_{\text{inc}}/k_{\text{mat}} = 2.0$. Later on, The phase contrast as well as BCs are also varied to investigate the PINN performance (see Section 3.2).

For the mechanical problem, we apply a tensile test while the upper and lower side of the specimen are traction free. For the thermal problem, we apply constant temperature at the left and right edge as shown in the right-hand side of Fig. 3. The upper and lower part are isolated. The material parameters as well as the model inputs are summarized in Table 1.

3. New architecture for PINNs

3.1. Mechanical problem

In the PINN, the output variables (i.e. displacement and/or stress) are approximated via fully connected feed-forward neural networks (FFNN) [16]. The network has three types of layers. The very first layer is the input layer, next we have several hidden layers, and finally comes the output layer [47]. Each layer can contain several neurons, and there is no connection between neurons inside a layer. Moreover, each layer is connected to the next layer to pass the information from the input layer to the output layer. For example, let us denote information within the lth

Table 1
Model input parameters for mechanical and thermal problems.

	Value/Unit
Mechanical problem (static equilibrium)	
Phase 1 Young's modulus and Poisson's ratio (E_{mat}, ν_{mat})	(0.5 GPa, 0.3)
Phase 2 Young's modulus and Poisson's ratio (E_{inc}, ν_{inc})	(1.0 GPa, 0.3)
Applied displacement in x-direction, $\bar{u}_x$	0.05 μm
Diffusion problem (heat conduction)	
Phase 1 heat conductivity (k_{mat})	1.0 W/m K
Phase 2 heat conductivity (k_{inc})	0.5 W/m K
Applied temperature (T_L, T_R)	(1.0 K, 0.0 K)

layer by the vector $\boldsymbol{z}^l$ where each component of the vector represents the value stored in that particular neuron. Therefore, transferring the information between two neighboring layers within a FFNN can be written as:

$$z_m^l = a(\sum_{n=1}^{N_l} w_{mn}^l z_n^{l-1} + b_m^l), \quad l = 1, \ldots, L. \tag{18}$$

Here the network has L layers and N_l neurons in the lth layer. The connection between the n-th neuron in layer $l-1$ and m-th neuron in layer l has a weight w_{mn}. For each neuron in the l th hidden layer, we have a bias term b_m^l. The input layer in this work is the components of the position vector in 2D, i.e. we have $z_1^0 = x$, and $z_2^0 = y$. Moreover, the output layer contains the desired solution and therefore is written as $z_1^L = u_x$, $z_2^L = u_y$, $z_3^L = \sigma_x$, $z_4^L = \sigma_y$ and $z_5^L = \sigma_{xy}$. Each neuron presents a summation operation (so-called weighted sum) plus a bias term and passes it through a non-linear activation function $a(\cdot)$. The activation function utilized in this work is Hyperbolic-Tangent (tanh) function [48]:

$$\tanh(x) = \frac{\exp^x - \exp^{-x}}{\exp^x + \exp^{-x}}. \tag{19}$$

The network trainable and input parameters are also shown by $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$ and $\boldsymbol{X} = \{x, y\}$, respectively. Here, $\boldsymbol{W}$ and $\boldsymbol{b}$ consist of all the weights and biases in neural network (their components for the l the layer are also shown in Eq. (18)). Finally, by denoting each neural network system by $\mathcal{N}$, we write the following solution components for the problem:

$$u_x = \mathcal{N}_{u_x}(\boldsymbol{X}; \boldsymbol{\theta}), \tag{20}$$
$$u_y = \mathcal{N}_{u_y}(\boldsymbol{X}; \boldsymbol{\theta}), \tag{21}$$
$$\sigma_x^O = \mathcal{N}_{\sigma_x}(\boldsymbol{X}; \boldsymbol{\theta}), \tag{22}$$
$$\sigma_y^O = \mathcal{N}_{\sigma_y}(\boldsymbol{X}; \boldsymbol{\theta}), \tag{23}$$
$$\sigma_{xy}^O = \mathcal{N}_{\sigma_{xy}}(\boldsymbol{X}; \boldsymbol{\theta}). \tag{24}$$

To build a given partial differential equation residual and its relative boundary conditions, one requires the differentiation of the output layer (e.g. displacement and stress) with respect to the input layer (e.g. position vector). In deep neural networks, the analytical differentiation is already available by using the automatic differentiation (AD) [49], which is a feature in modern deep learning frameworks such as Pytorch [50] and Tensorflow [51].

Based on the described set of equations for the mechanical problem (Eqs. (1) to (5) as well as Eqs. (9) and (10)), the following loss functions are defined below. As discussed earlier, the new architecture consists of three main parts. First, we have the loss terms applied to the displacement vector $\boldsymbol{u}$. These terms are based on the energy form of the problem as well as the Dirichlet BCs ($\mathcal{L}_{EF}$ and $\mathcal{L}_{DBC}$). Next, we add the loss terms applied to the stress tensor $\boldsymbol{\sigma}^o$ as well as the Neumann BCs ($\mathcal{L}_{SF}$ and $\mathcal{L}_{NBC}$). Finally, we have the connection loss term $\mathcal{L}_{cnc}$.

$$\mathcal{L} = \underbrace{\mathcal{L}_{EF} + \mathcal{L}_{DBC}}_{\text{based on } \boldsymbol{u}} + \mathcal{L}_{cnc} + \underbrace{\mathcal{L}_{SF} + \mathcal{L}_{NBC}}_{\text{based on } \boldsymbol{\sigma}^o}, \tag{25}$$

$$\mathcal{L}_{EF} = \text{MAE}_{EF}\left(-\int_\Omega \frac{1}{2}\boldsymbol{\sigma} : \boldsymbol{\varepsilon}\, dV + \int_\Gamma (\boldsymbol{\sigma} \cdot \boldsymbol{n})\boldsymbol{u}\, dA\right), \tag{26}$$

$$\mathcal{L}_{DBC} = \text{MSE}_{DBC}\left(\boldsymbol{u} - \overline{\boldsymbol{u}}\right), \tag{27}$$

$$\mathcal{L}_{cnc} = \text{MSE}_{cnc}\left(\boldsymbol{\sigma}^{o} - \boldsymbol{\sigma}\right), \tag{28}$$

$$\mathcal{L}_{SF} = \text{MSE}_{SF}\left(\text{div}(\boldsymbol{\sigma}^{o})\right), \tag{29}$$

$$\mathcal{L}_{NBC} = \text{MSE}_{NBC}\left(\boldsymbol{\sigma}^{o} \cdot \boldsymbol{n} - \overline{\boldsymbol{t}}\right). \tag{30}$$

The energy loss (Eq. (26)) is supposed to be minimized and it takes at the global response of the system, whereas the other loss terms are satisfied locally for each collocation point. Furthermore, we have the definition of the mean squared error according to

$$\text{MSE}(\bullet)_{type} = \frac{1}{k_{type}} \sum_{i=1}^{k_{type}} (\bullet)^2, \quad \text{MAE}(\bullet)_{type} = \frac{1}{k_{type}} \sum_{i=1}^{k_{type}} |\bullet|, \tag{31}$$

$$k_{EF} = 1, \quad k_{DBC} = N_{DBC}, \quad k_{cnc} = N_b, \quad k_{SF} = N_b, \quad k_{NBC} = N_{NBC}. \tag{32}$$

Here, k_{type} is the number of collocation points for each loss term. Note that, we have different number of collocation points for each boundary type (i.e. N_{DBC} and N_{NBC}) and bulk (N_b). The above relations are further exploited in Appendix C for described BVPs in Fig. 3.

The solution to the mechanical problem is calculated by minimizing the total loss function in Eq. (25) on the set of collocation points. The final optimization reads as:

$$\boldsymbol{\theta}^{*} = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{X}; \boldsymbol{\theta}). \tag{33}$$

In the above equation, $\boldsymbol{\theta}^{*}$ includes the optimal trainable network parameters. The collocation points $\boldsymbol{X}$ are located inside the domain and on the boundary.

In this work, the Adam optimizer [52] is employed. Several parameters and hyper-parameters affect the network's performance, such as activation function, learning rate, number of epochs, batch size, number of hidden layers, and number of neurons in each layer. Their influence should be investigated based on the number of collocation points and the network's architecture. This matter is explained in more detail in Section 3.1.2.

Remark 1. One may consider different weightings for the terms in Eq. (25). Our preliminary studies show that in some cases, by tuning the weightings, one can improve the results for the same number of iterations in training. In the current work, we considered equal weightings.

Remark 2. In Eq. (26) we considered mean absolute error (MAE) while for the other terms, mean square error (MSE) is utilized. Note that in the energy formulation, the magnitude of the loss term is much smaller compared to other terms (see the multiplication of stress and strain tensor). According to our investigation and as it is discussed in Section 3.1.1, using MAE for the energy term is more beneficial to keeping the values of different loss functions in the same order.

The architecture of the network for the mechanical problem is shown in Fig. 4. The network is based on the Eqs. (25) to (30) which shows up on the right-hand side of the figure. The proposed network in this study shares some similarities with the recent and previous investigation in the literature [16,17,19,28,31]. Nevertheless, it also differs from them in the following aspects. First, the network takes components of the position vector as inputs (x and y coordinate in a 2D setting) and predicts the components of the deformation vector $\boldsymbol{u}$ and stress tensor $\boldsymbol{\sigma}^{O}$ as outputs. Using the predicted deformation vector components (u_x and u_y) and based on the kinematic relation (Eq. (1)) and constitutive law (Eq. (2)), strain and stress tensors are calculated from automatic differentiation, respectively. The latter quantities are denoted by $\boldsymbol{\varepsilon}$ and $\boldsymbol{\sigma}$. This stress tensor is evaluated from deformation $\boldsymbol{u}$ and it is in principle different from the stress tensor $\boldsymbol{\sigma}^{O}$ which is the direct output of the network. Later on, these quantities are forced to take the same value utilizing the connection loss term $\mathcal{L}_{cnc}$ presented in Eq. (28). Next, in the new design, we consider separate networks for each of the output variables. According to our investigations which will be reported later on in this work, having separated networks for each output quantity shows some advantages compared to the case where all the networks are combined.

Note that for constructing the physical constraints (i.e. loss function) we only need first-order derivatives of the output variables with respect to the input parameters. Reducing the order of derivatives also seems to be beneficial

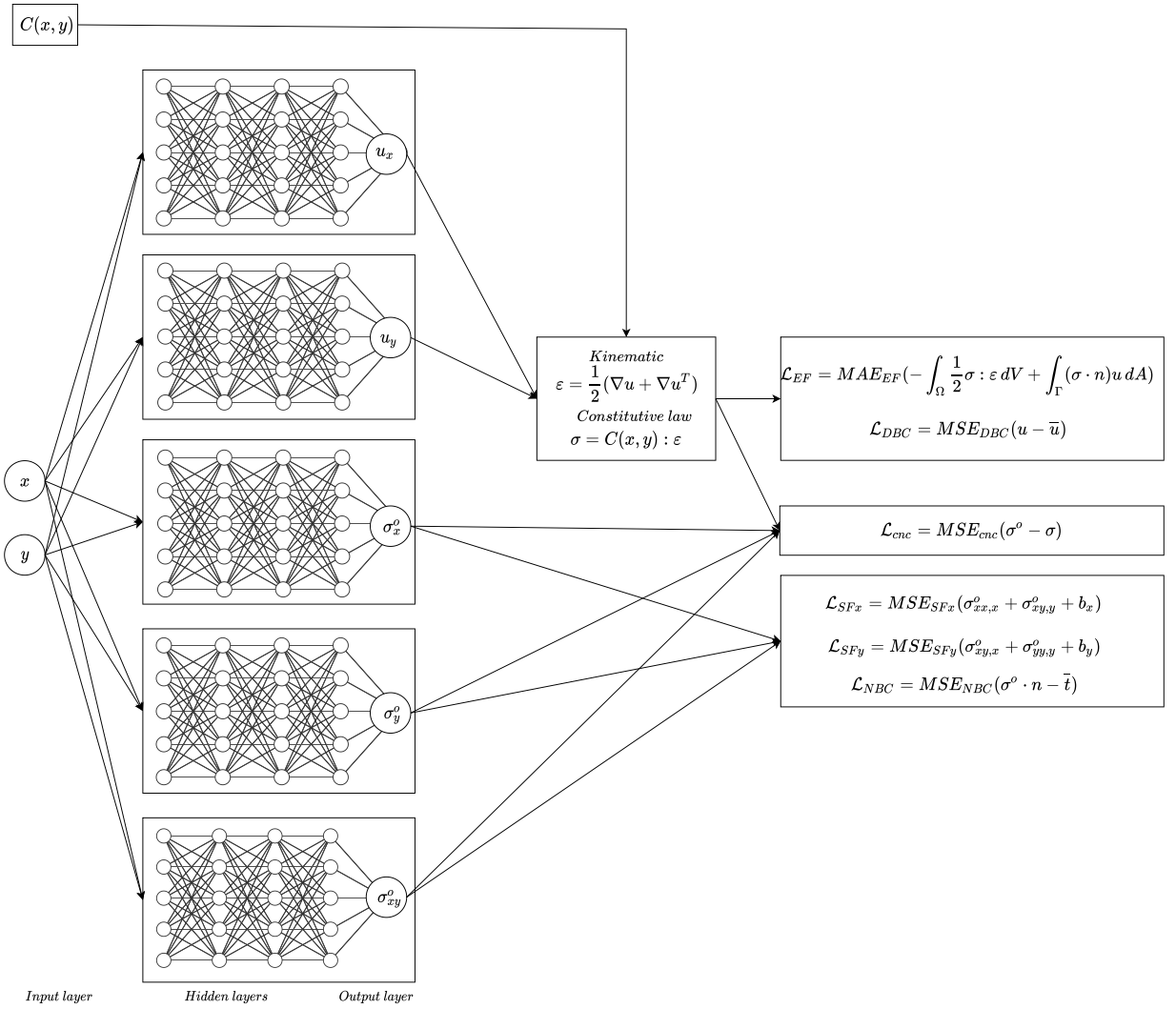

Fig. 4. Architecture and loss functions for the mixed formulation based network in the mechanical problem.

in different aspects. First, one has more flexibility in choosing the activation functions (i.e. we avoid vanishing its gradients by taking higher-order derivatives). Second, the computational cost is decreased compared to the cases where we have second-order derivation. Moreover, as we will show later on, it seems that lower-order derivatives result in more accurate predictions (less overall error). In simple studies in Appendix B, we indicate that the lower the degree of derivation is, the easier it is for the network to find the solution for a given number of iterations and collocation points.

The proposed network's design is achieved by performing and comparing several computations for different network architectures. The summary of considered models (network architectures) is depicted in Table 2. The current work is mainly based on model E in which multiple outputs for displacement and stress quantities exist. Note that in models B, C and E only first-order derivatives of the outputs are required for obtaining the loss functions.

The network parameters are summarized in Table 3. For other models, similar parameters are utilized except for the architecture of the inputs and outputs which is according to Table 2. In addition to the learning rate parameter α, we have other parameters for the better stability and convergence of the Adam optimizer. The first moment is set to 0.9 while the second moment of the gradient is set to 0.999 (see also [52]).

Table 2
Summary of possible models (network architectures) which can be used for PINNs. The inputs are always the same (position), while outputs and loss functions (LFs) are changed concerning the architecture. The current work is based on model E.

Name	Description
Model A	outputs: components of $\boldsymbol{u}$, LFs: governing PDE and BCs, separated NNs are used
Model B	outputs: components of $\boldsymbol{u}$, LFs: energy form of the PDE and BCs, separated NNs are used
Model C	outputs: components of $\boldsymbol{u}$ and $\boldsymbol{\sigma}$, LFs: governing PDE and its energy form, combined NN is used
Model D	outputs: components of $\boldsymbol{u}$ and $\boldsymbol{\sigma}$, LFs: governing PDE, separated NNs are used
Model E	outputs: components of $\boldsymbol{u}$ and $\boldsymbol{\sigma}$, LFs: governing PDE and its energy form, separated NNs are used

Table 3
Summary of the network parameters.

Parameter	Value
Inputs, outputs	$\boldsymbol{X}$, $(\boldsymbol{u}, \boldsymbol{\sigma}^O)$
Activation function	tanh
Number of layers and neurons per layer (L, N_l)	(5, 40)
Batch size	full batch
(Learning rate α, number of epochs)	$(10^{-3}, 10^5)$

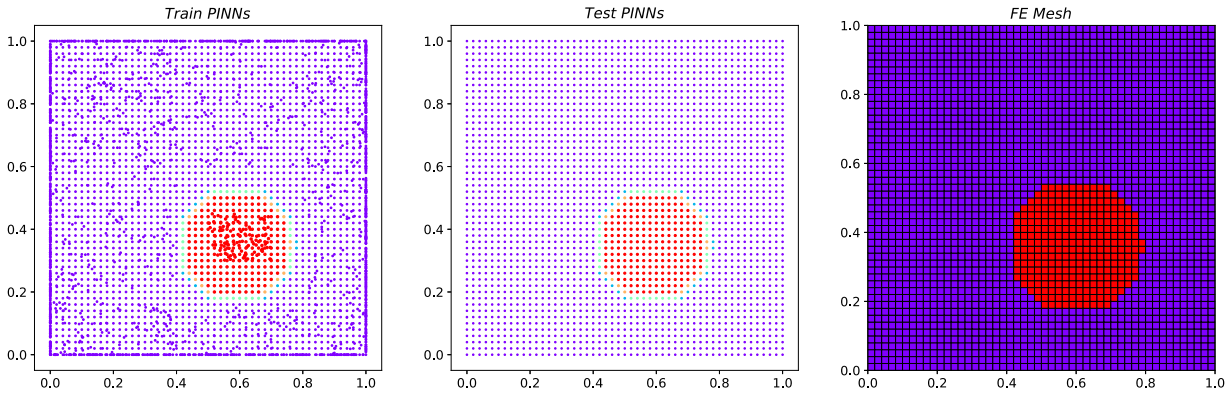

Fig. 5. Left: collocation points utilized in the introduced PINN solver. Middle: points at which the trained network is evaluated. Right: quadrilateral meshes to discretize and solve the problem with finite element method. Different colors represent changes in the Young's modulus of the material.

3.1.1. Single heterogeneity in a matrix (geometry 1)

To train the network (i.e. minimizing the introduced loss functions), collocation points are defined according to the left-hand side of Fig. 5. As we will discuss later, to improve the results, we consider more points (in a random way) within the red phase and the purple phase. Later on we evaluate the trained network for the test points shown in the middle of Fig. 5. The total number of collocation points is around 5000. On each boundary, we considered 400 points. Note that for a better comparison between FE and PINN, the FE meshes are defined regularly. Moreover, the same number of elements is considered for FEM (on the right-hand side) as points for PINN (in the middle of the figure). Colors indicate Young's modulus value (see Table 1). The Young's modulus of the points at the interface is computed concerning neighboring elements materials in FE mesh. This strategy is utilized to mimic the assembly procedure of FEM and assigns the relevant stiffness to every point. Different loss terms introduced earlier are now minimized with respect to free parameters of the network (i.e. weighting and biases). In Fig. 6, all the loss terms are plotted versus the number of epochs (iterations for the training).

We minimized the same problem (same loss functions) for five different times to make sure that the introduced network is always able to minimize the total loss function. The main loss terms and how they decay through the training are plotted individually in Fig. 6. We observe a very small values for the loss regarding the energy term that corresponds to MSE formulation. As we checked, by using absolute value error, the loss value will be in order

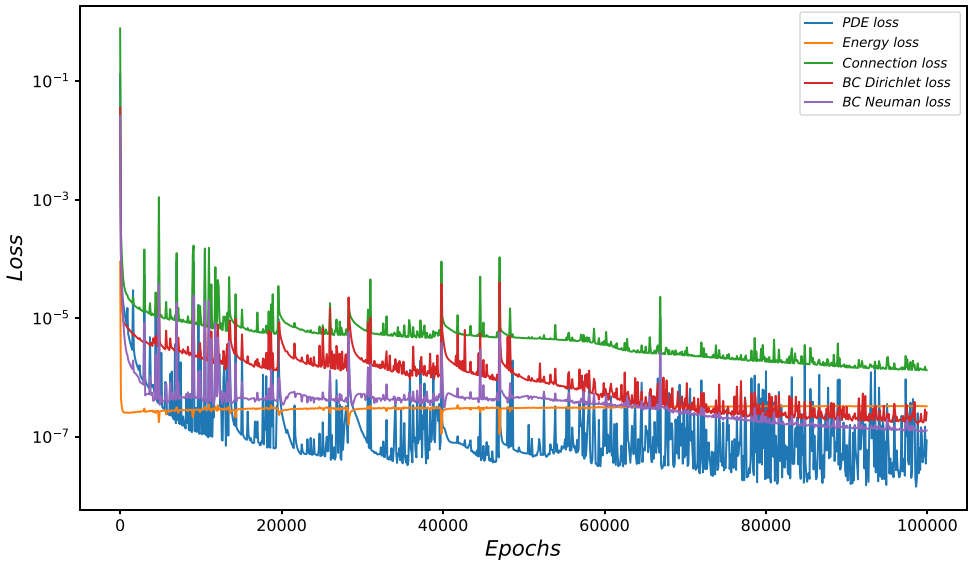

Fig. 6. Individual loss terms versus number of epoch for geometry 1 using model E.

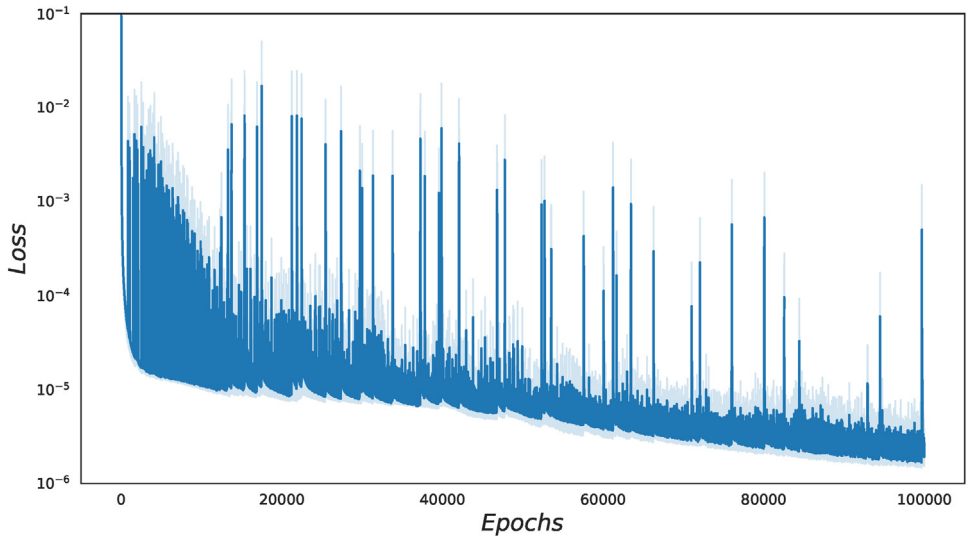

Fig. 7. The total loss term together with the confidence bound 95% for geometry 1 using model E.

of other loss terms but the final results did not changed significantly. The total loss value and its performance with 95% confidence bound are plotted in Fig. 7.

We used Skylake CPU, Platinum 8160 model, and Intel HNS2600BPB Hardware Node Type. The computation for each BVP on averaged took around 2 h, which is longer than the computation time from a FEM in the comparable hardware. This time can be reduced by using the GPU power, and it is not in our current scope to further elaborate on this point.

The deformed configurations from the PINNs and the FEM are presented in Fig. 8 where the deformations are magnified by a factor of 10. The result match very well despite the small violation of boundary conditions on the left edge. Further results are reported in Fig. 9 wherein the first two rows are the predicted displacement vector components and in the last three, the stress tensor components are presented. In the first, second, and third columns, the results from PINNs, FE, and the difference between these two methodologies are presented, respectively.

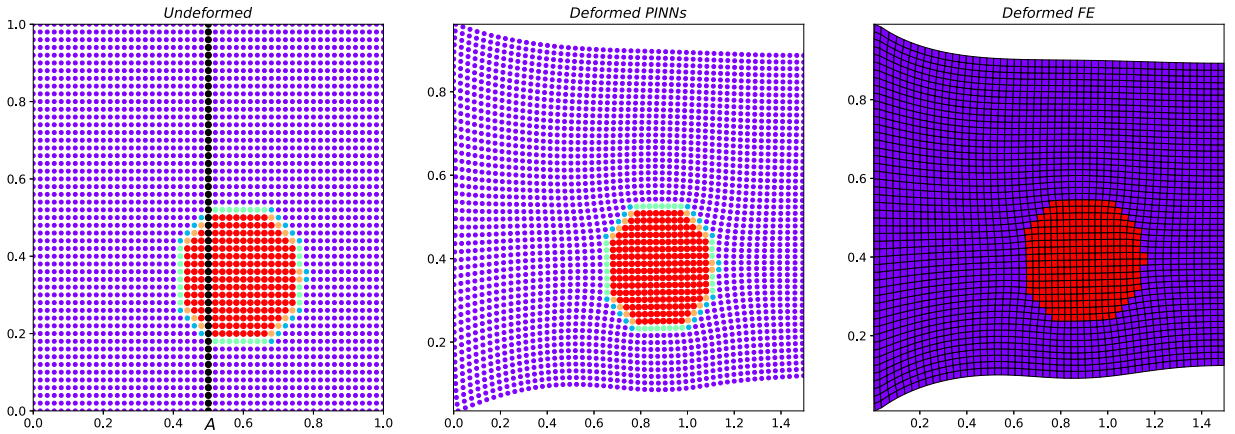

Fig. 8. Undeformed and deformed configurations obtained from the mixed formulation based PINNs and FEM for the mechanical problem considering geometry 1. The results are magnified by a factor of 10.

In overall, for all the output values, we observe a good agreement between predictions of the network and the FE method. The difference between the results is bold first near to the corner points at the left boundary, and second, in the area where we have the sharp phase transition. The maximum relative difference which is localized in the critical spots are about 44% and 77% for displacement and stress, respectively. The averaged relative difference value is about 4% and 3.7% for displacement and stress values, respectively. Despite the acceptable performance for the normal components of the stress tensor, the prediction of the mixed formulation based PINNs for the shear component (σ_{xy}) is relatively poor. As we discuss in Section 3.5 and Appendix B, the difference can be reduced by some simple remedies such as running the PINNs model for more epochs and adding more initial collocation points in critical spots such as the external boundary of the problem and interphase regions.

One may notice that for this example, the chosen points for evaluating the results and comparing them against those from FEM are shown in the middle part of Fig. 5. These points are mainly included in the collocation points (left side of Fig. 5). In the next study, we will examine this matter further for higher resolution (more test points and meshes) to make sure the obtained solution can be properly interpolated in the whole domain.

According to Table 2, one has different options for designing the network architecture. In models A and D, the PDE is directly implied in the loss function. In these cases, we expect second-order derivation. The latter point can be a drawback as the network has to approximate the solution by taking higher order of derivatives compared to the previous case. In Appendix B, we show in a simple example that the lower the degree of derivation is, the easier it is for the network to find the solution. Therefore, utilizing the energy form of the problem can be beneficial (see also [28]). In models A and B, the only output of the network is the displacements $\boldsymbol{u}$ whereas in the rest of the models, components of the stress tensor are also considered as additional outputs. We would like to study this point and argue that having an additional set of outputs helps for better training in heterogeneous systems (see also [28,31,33]). In model E (current work), in addition to separate outputs for displacements and stresses, also a separate network is proposed for the each output [31]. To show the performance of each model, we utilized each of them on geometry 1 under the same boundary conditions. For each training, 100K epoch is used. The results for predicted u_x, u_y, σ_x, and σ_y along the cross-section at $x = 0.5$ μm are shown in Fig. 10.

Considering the results from the FE method as the reference solution, only models C and E were able to capture the response of the heterogeneous system properly. The other models were able to capture the overall behavior but many details are missed. The results of model B compared to A indicates that only reducing the derivation order is beneficial (see also Appendix B). Note that, in model B, we only minimize the total energy globally, while in model A, the governing equations are satisfied pointwise. The latter argument can be an explanation for errors in model B. Comparing models A and D indicates that by only increasing the number of outputs, one may not achieve a significant improvement. On the other hand, by combining all these refinements together, more accurate response is achieved. The performance of model C seems to be less accurate compared to model E which indicates the advantage of using a separated network for each of the output variables. Note that utilizing separate NN for

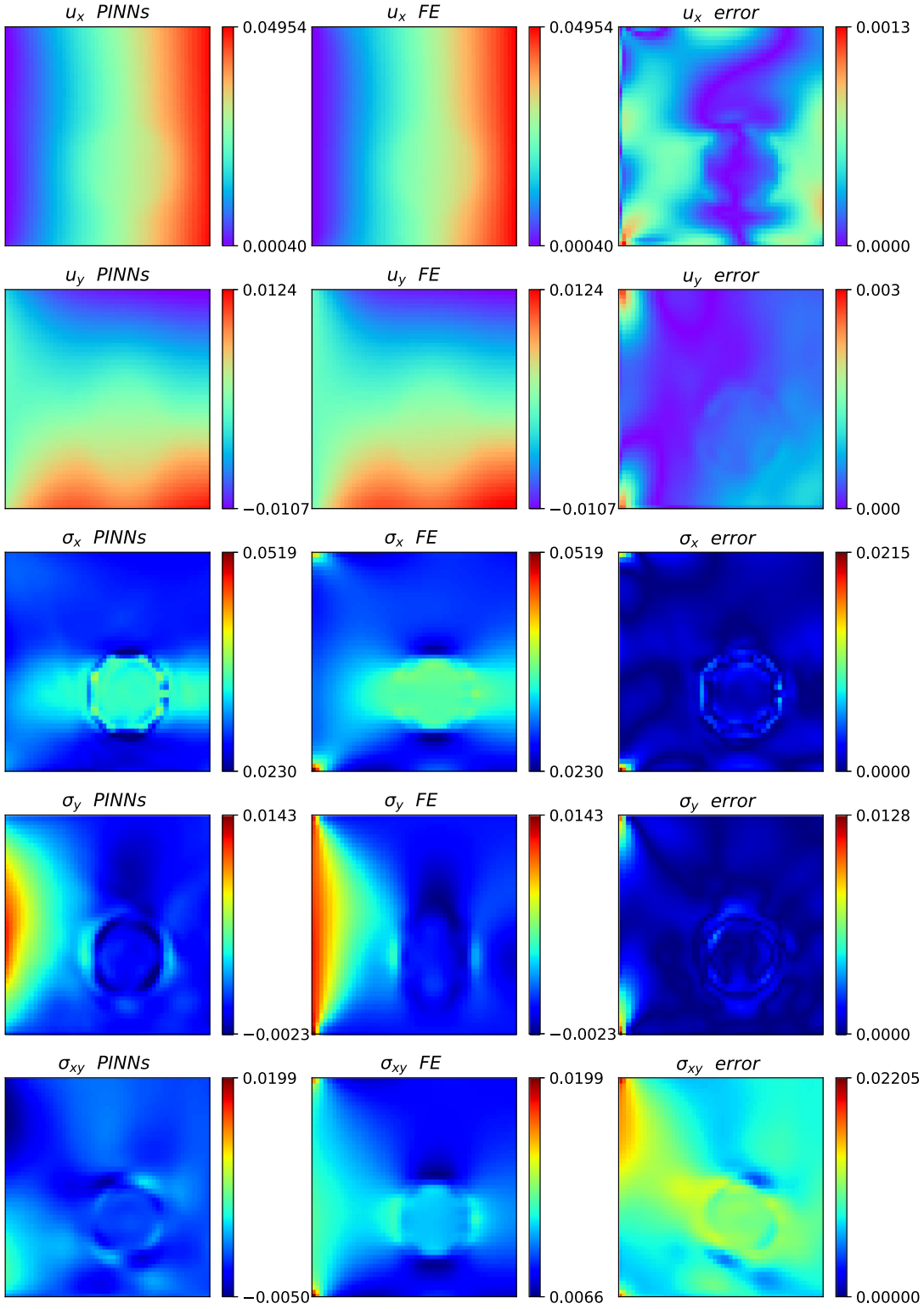

Fig. 9. Obtained results for the mechanical problem using geometry 1 and the mixed formulation based PINNs and FEM.

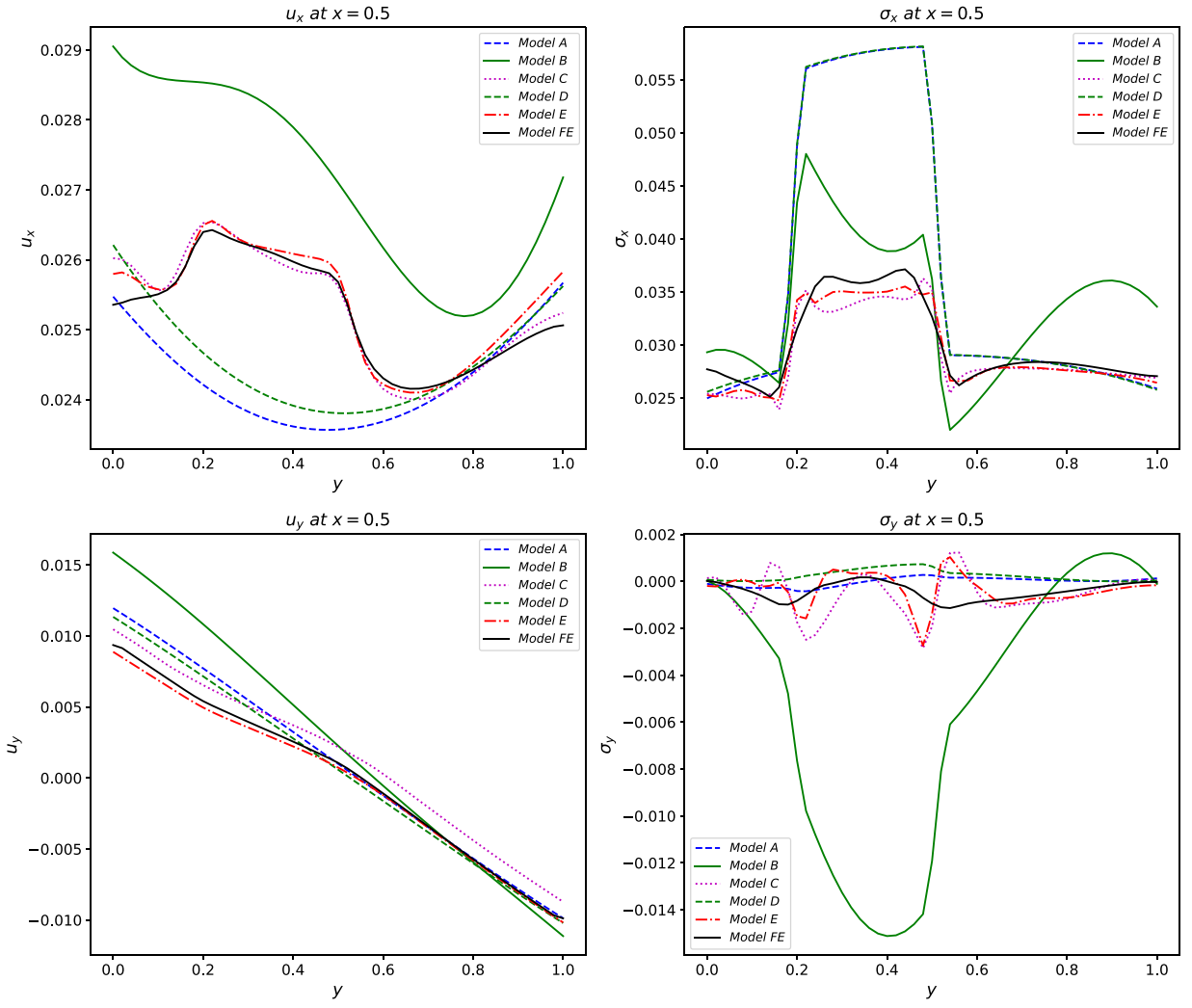

Fig. 10. Comparison between obtained results from examined networks and FE along $x = 0.5$ μm direction. For Model A, outputs are components of $\boldsymbol{u}$ and loss functions are based on governing PDE as well as BCs. For Model B, outputs are components of $\boldsymbol{u}$ and loss functions are based on energy form of the PDE as well as BCs. For Model C, outputs are components of $\boldsymbol{u}$ and $\boldsymbol{\sigma}$ and loss functions are based on governing PDE and its energy form in a combined NN. For Model D, outputs are components of $\boldsymbol{u}$ and $\boldsymbol{\sigma}$ and loss functions are based on governing PDE where separated NNs are used. For Model E, outputs are components of $\boldsymbol{u}$ and $\boldsymbol{\sigma}$ and loss functions are based on governing PDE as well as its energy form where separated NNs are used.

each output means that we used much more (roughly five times more) free variables for the network's training. In the future, more studies are required to investigate the optimum design of the network architecture.

Based on the results in Fig. 10, we conclude that on average, the difference between the network predictions and the FE solution is higher for the stress values. Nevertheless, model E managed to capture the jump in the response properly. We believe that part of the difference is also because of the way we meshed the system for FE. In the future comparison, one can also employ a better discretization for the interface of the different phases. In the current work, we intend to stay with the so-called pixel-based mesh which might introduce unnecessary jumps in the prediction of the FE results. See the sharp corners in Fig. 5 for the elements at the border of heterogeneity.

Remark 3. The above comparison does not imply that models A, B, and D are not useful for finding the solution in a heterogeneous domain. One can do further studies and tune the number of neurons, layers, the number of epochs, and collocation points accordingly to get a better response out of each model. This also holds for model C. In

general, having a fair comparison between all these models is not straightforward. The least we can conclude is that adding separate networks combined with the energy form can be beneficial for finding the solution in heterogeneous domains. For a better understanding readers are also encouraged to see Appendix B, where we performed studies on a simplified ordinary differential equations.

3.1.2. Discussions on the parameters and hyper-parameters for deep learning model

In feed-forward neural networks, there are several training parameters and hyper-parameters which affect the optimization process and also the final value of the loss function. Therefore, to get the best performance from the neural network, one requires to perform a series of parameter studies during the training process. In the following we summarize our experience with choice of some hyperparameters.

- Activation functions have a significant influence on the network performance. Depending on the order of the PDE, the higher-order derivative of the active functions should be non-zero for the gradient descent algorithm to perform well. In this work, we tried various qualified activation functions such as *tanh*, *sigmoid*, and *softplus*. From available options, *tanh* showed the best performance in terms of the final value of the loss function.
- It is crucial to use sufficient epochs. In the process of training, when the network sees all of the collocation points, then the network is trained for one epoch. One can also define a criterion to stop the training based on the total loss. However, in this work, we set the stopping value as 10^{-12}, and we train our networks for 10^5 epochs.
- We can feed all the collocation points for the network training at the same time in one step, or we can divide them into several batches or mini-batches. By choosing mini-batches, we have the option to shuffle the data in each epoch. By using a lot of mini-batches, the computational time usually increases because the network requires more iterations for each epoch. However, using a full-batch we get faster training while it requires more memory to store all the collocation points for one epoch. In this study, we used a full batch for our training. One main reason is the restriction from the energy loss term where we need to integrate over all the collocation points to compute internal energy.
- The value of the learning rate is normally between $[1e{-}4,\ 1]$, see [53]. It is also possible to choose an adaptive learning rate based on the value of the loss function, which leads to more intelligent convergence. See also discussions in Appendix A for further possibilities on using a more complex form of learning rate.
- The number of hidden layers and the number of neurons should be also studied. Having a higher number of layers (i.e. a deeper network) increases the number of neurons and trainable parameters. Choices for these two parameters severely depend on the nature of the problem and to some extent the other parameters which have been mentioned. Therefore, the selected numbers should be studied based on the number of collocation points for each training. We kept the number of neurons in each layer equal, and after several studies, we finally chose 5 layers with 40 neurons in each layer. The criteria for preferring this network are: firstly, the final value of the total loss function at the end of the training, and secondly, the absolute error of the output values. Meanwhile, by increasing the number of neurons, overfitting may occur [54].
- The number and location of collocation points play an important role. According to discussions in Appendix B and Section 3.5, adding more collocation points (specially in regions where we expect high gradients of variable) helps to obtain more accurate solutions. This is also similar to the location of nodes and element refinement in FEM.

3.1.3. A more complex case for heterogeneity

To show the applicability of the mixed formulation (model E), we examine a more complex pattern for heterogeneity. For this case, the collocation points are shown on the left-hand side of Fig. 11. Here the total number of collocation points is around 5000. On each boundary, we considered 400 points. In the middle part of Fig. 11, the points, where the trained network is evaluated are shown. The network is evaluated in a much denser grid (100×100 points) where the majority of these points are not located in the set for collocation points. By such selection, we intend to show the power of interpolation in PINNs within the domain of study. The FE approach is also performed on a very fine mesh (100×100 elements) shown on the right of Fig. 11. The undeformed and deformed shape of the microstructure is shown in Fig. 12. Further results of the study are summarized in Fig. 13. Despite the complex shape of heterogeneity, the network can predict the distribution of deformation and stress components. Again, we observe that the main difference is mostly at the corners of the left boundary.

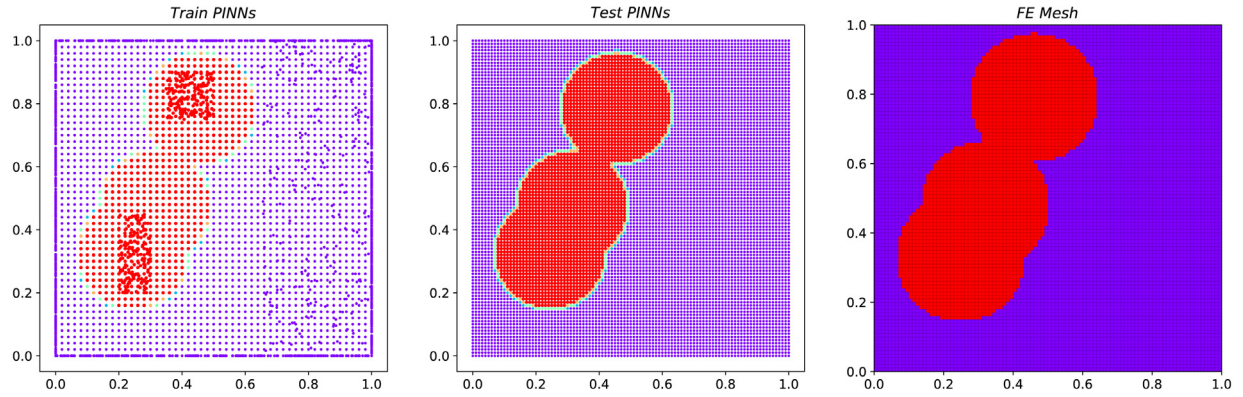

Fig. 11. Left: collocation points for the network training. Middle: points at which the trained networks is evaluated. Right: quadrilateral meshes to for FEM. Different colors represent changes in the Young's modulus.

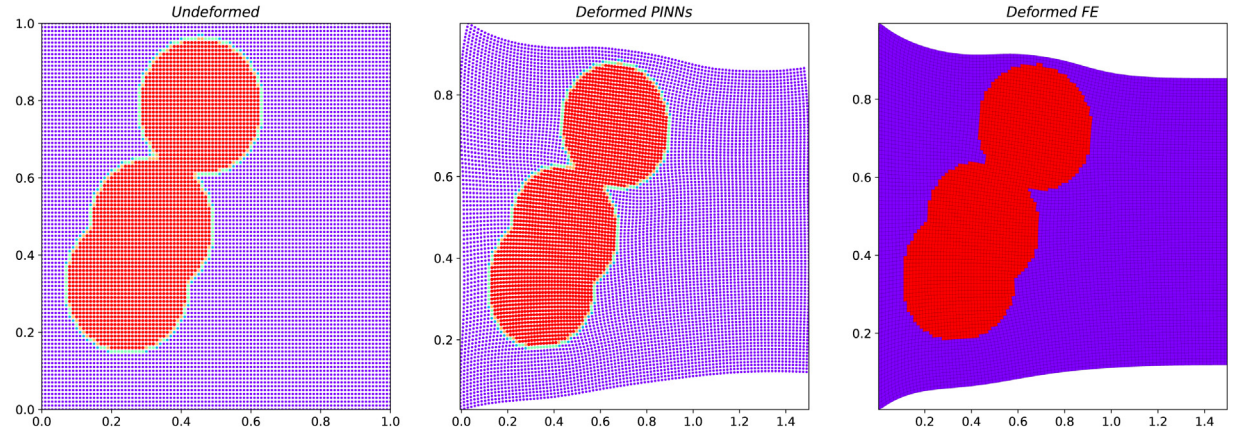

Fig. 12. Undeformed and deformed configuration obtained from the mixed formulation based PINNs and FEM for the mechanical problem considering geometry 2. The results are magnified by a factor of 10.

3.2. Other type of BCs and material properties

The mixed formulation based model is trained for simulation of other BCs and material properties. Here, we took the same network described in Section 3.1. On the right edge we applied Dirichlet BCs for both displacement components, i.e. $\bar{u}_x = \bar{u}_y = 0.05$ μm. The left boundary is completely fixed, and the top and bottom edges are traction free. Moreover, we increase the contrast in the Young's modulus of the phases to a higher number, i.e. $E_{\text{inc}}/E_{\text{mat}} = 10$. The obtained deformed structures utilizing PINNs and FEM are shown in Fig. 14.

For a quantitative comparison, displacement as well as stress in x direction at $x = 0.5$ μm are plotted along the y axis in Fig. 15. For the displacement field, we observe a close response between the two approaches except for the error at the boundaries for the mixed formulation based PINNs. For the stress profile, one observes more deviation between the two, but the overall response is well represented. Further comparisons are presented in Fig. 16. Here, the values of σ_{xy} are higher due to the BCs and in good agreement with FE results. The maximum error for stress components is about 10%, and it is localized close to the outer and phase boundaries.

3.3. Combination of data and physics for unseen prediction

The above investigations show the potential of the mixed PINNs formulation for obtaining the solution. Utilizing a standard optimizer such as Adam, one requires relatively higher computational time to find the solution for a given BVP compared to standard methods such as FE. Nevertheless, the DL methods have the potential to grow and get

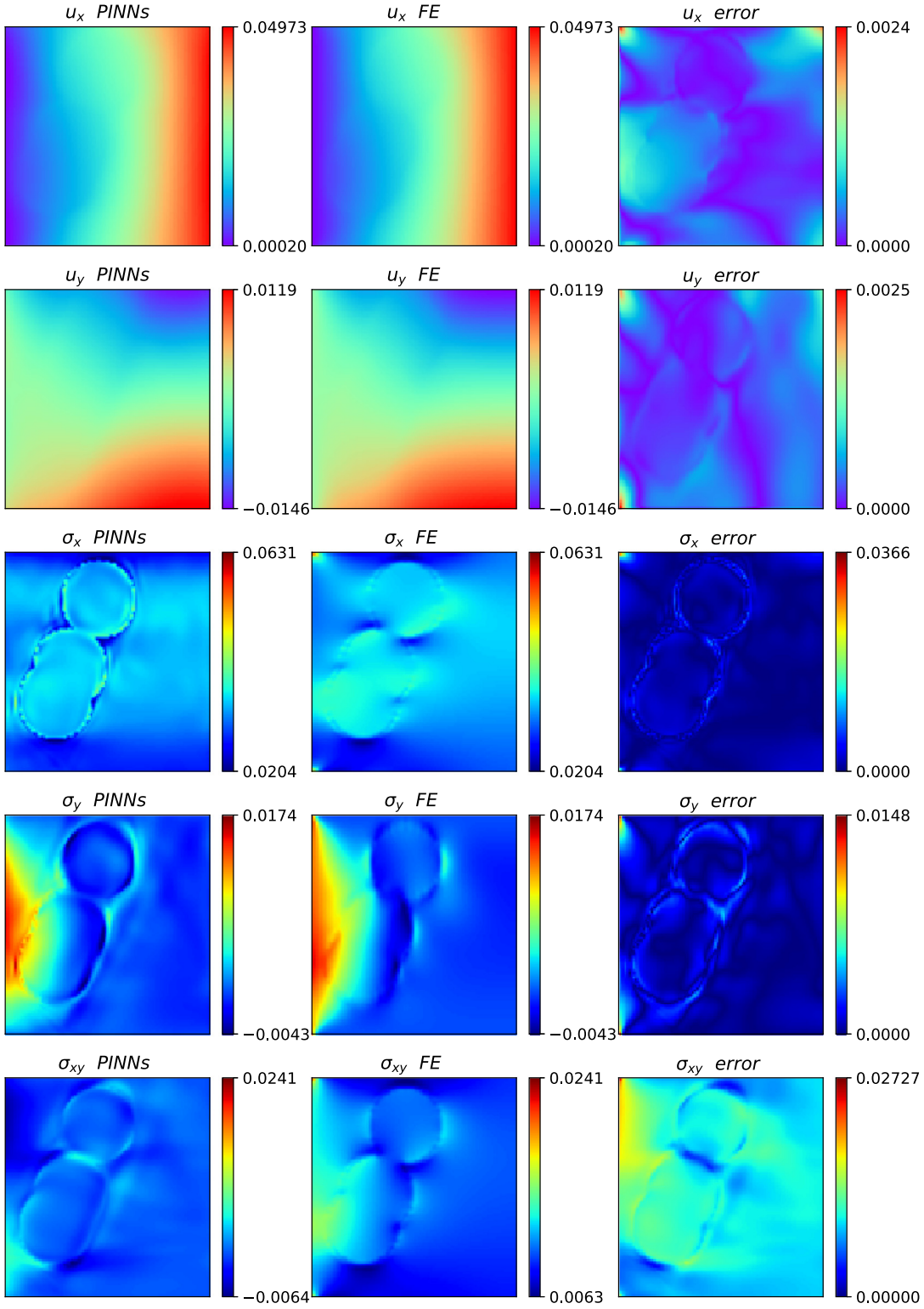

Fig. 13. Results for the mechanical problem using geometry 2 and the introduced mixed formulation based PINNs and the FEM.

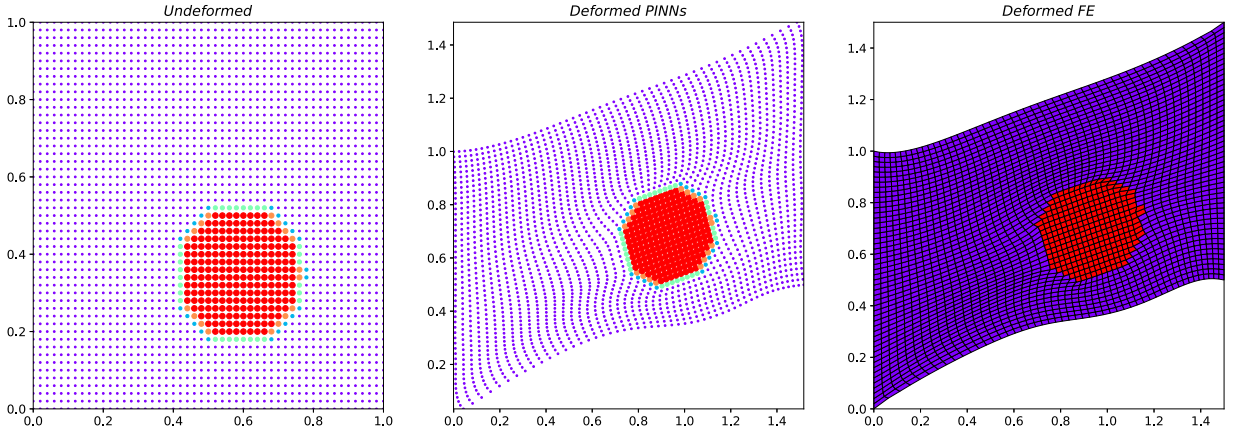

Fig. 14. Undeformed and deformed configuration obtained from the introduced mixed formulation based PINNs and FEM considering geometry 1. The results are magnified by a factor of 10.

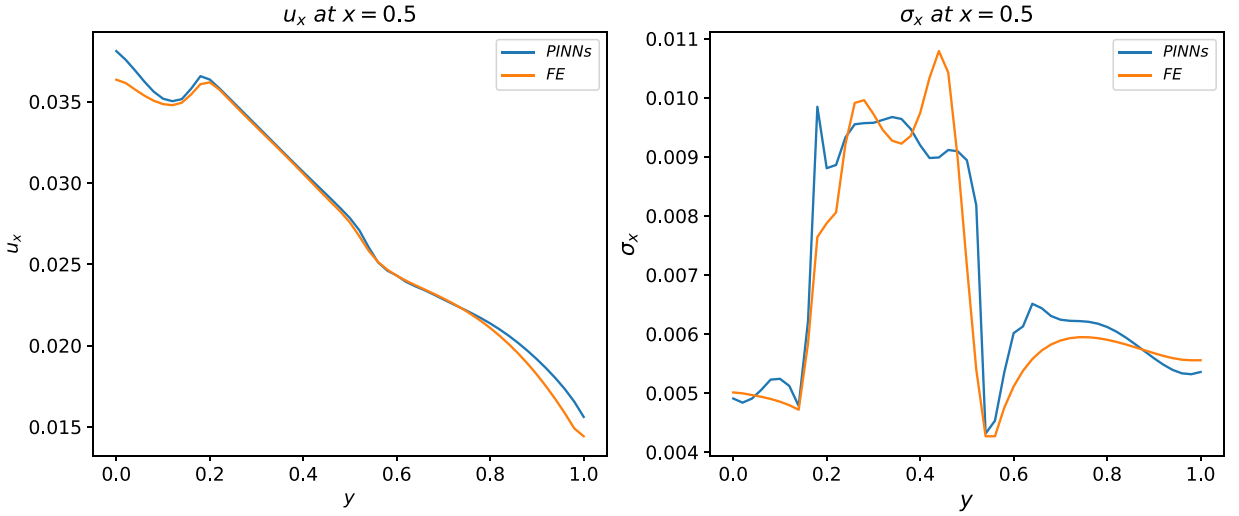

Fig. 15. Comparison between the mixed formulation based PINNs and FE along $x = 0.5$ μm direction for geometry 1.

trained based on other free parameters in the system. As a result, for new problems we expect to obtain the solution immediately (fraction of a second) which is much faster than any FE solver where we need to repeat the calculations as we change some initial parameters in the system. For better and faster training, we intend to encourage the use of the solution of the FE methods for some given cases as well. Therefore, the main idea here is to combine the data from FE with the physical constraints to train the network for further unseen predictions.

Here, we took the same example from the previous section and start to change Young's modulus ratio between the two phases (i.e. $E_r = E_{\text{mat}}/E_{\text{inc}}$). In this new study, the parameter E_r is set as an additional input for the network. FE calculations are then performed for 10 different cases. i.e. $E_r = \{1.0,\ 0.9,\ 0.8,\ 0.7,\ 0.6,\ 0.5,\ 0.4,\ 0.3,\ 0.2,\ 0.1\}$. We start with the first input ($E_r = 1.0$) and train the network shown in Fig. 17 and Eq. (34) for 10 000 epochs. Next, we take the trained network (optimized weights and biases) from the previous step and change the second input to $E_r = 0.9$. The network is then re-trained for 10 000 epochs. The computational cost for each training is about 40 min. The same patterns will be repeated for the rest of the cases for E_r. Note that as shown in Eq. (34), one can tune the contribution of the physical loss functions via the parameter $\omega \geq 0$. According to our early results, this parameter shall be tuned for better training and more investigations shall be devoted to this point in future studies. For the current studies we set $\omega = 0.1$. Recent investigations also address the self-adaptive weighting algorithms

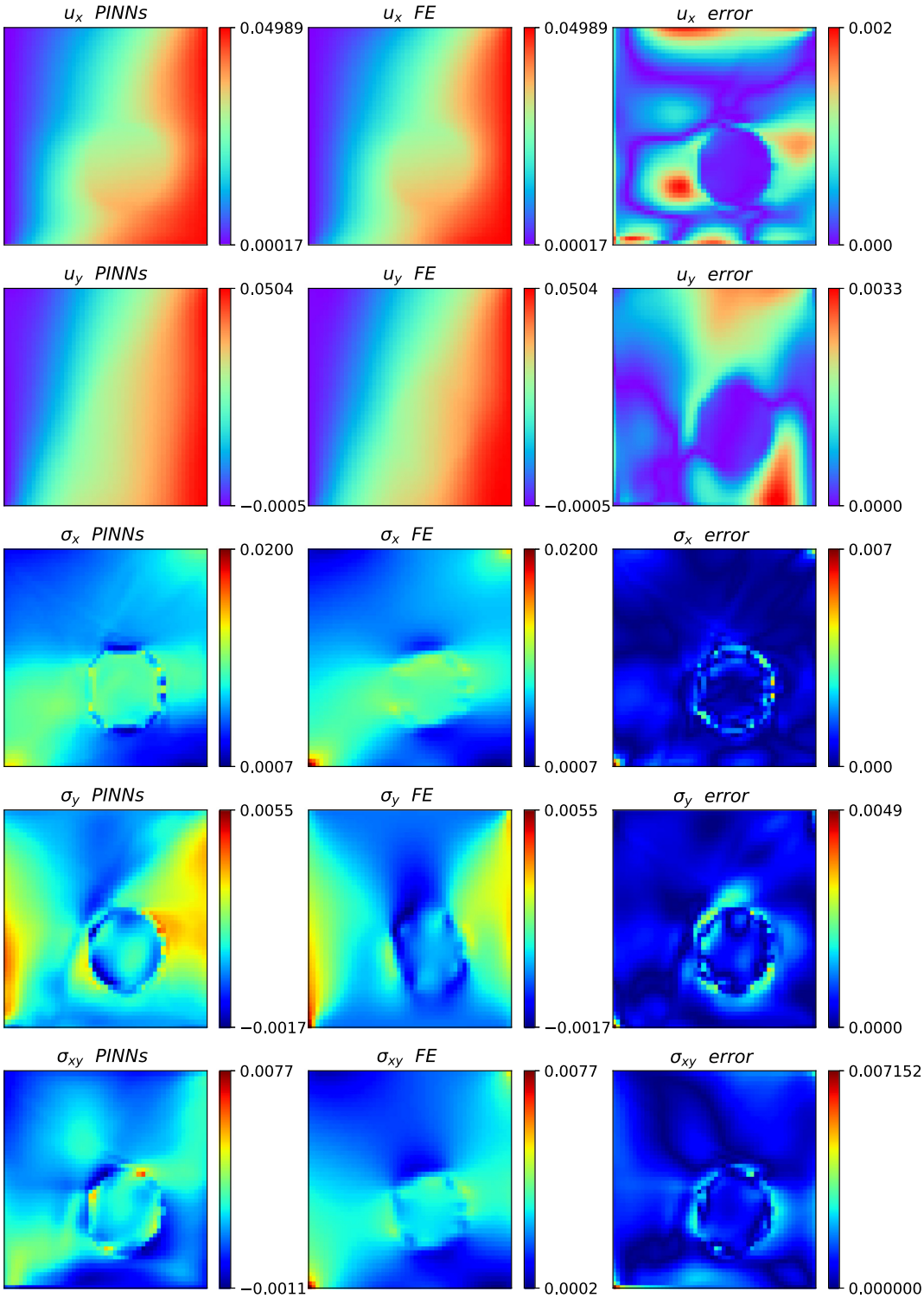

Fig. 16. Results for the mechanical problem using new boundary conditions ($\bar{u}_x = \bar{u}_y = 0.05$ μm) and the new phase contrast ($E_{\text{inc}}/E_{\text{mat}} = 10$).

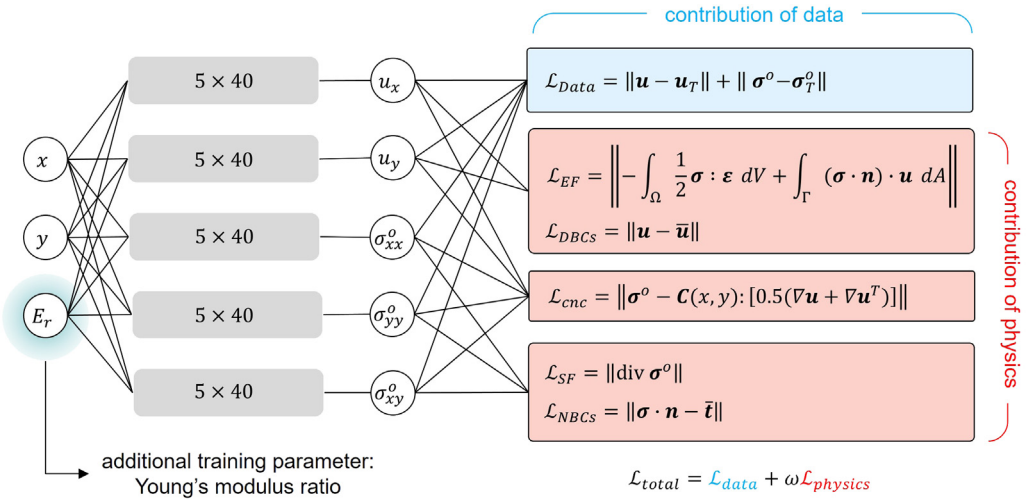

Fig. 17. New architecture for the mixed PINNs where we have the contribution of data and physics. The neural network model predict the solution for any arbitrary ratio for the Young's modulus $E_r = E_{\text{mat}}/E_{\text{inc}}$. The weighting parameter ω shows the contribution of the physical loss functions to the total loss function.

for this purpose [55].

$$\mathcal{L}_{total} = \underbrace{\mathcal{L}_{data}}_{\text{FE calculation}} + \omega \underbrace{\mathcal{L}_{physics}}_{\text{given BVP}}. \tag{34}$$

In Fig. 17, the loss term related to data ($\mathcal{L}_{data}$) is the difference between the predicted values from the network and those from FE calculations for that particular Young's modulus ratio E_r. The physical loss terms are exactly same as before (see Eqs. (25) to (30)). The other network parameters remain similar to the previous section. To check the performance of the physics-data-driven network, we present here one rather extreme case which is not included in the data and is beyond the range of the training set, i.e. $E_r = 1/12 \approx 0.083$. This case shows the power of the network for the extrapolation and the results of the predictions are reported in Fig. 18.

The average relative difference for u_x is about 0.00034. This difference for σ_x and σ_y reads 0.001 and 0.00014, respectively. At this point, it is interesting to look into the network prediction which is solely trained based on the data from FE without any applied physical constraints (i.e. $\omega = 0.0$). The results are presented in Fig. 19. Comparing different components in Figs. 19 and 18, the overall predictions for the stress components improved significantly by adding the contribution of governing equations. As an example, the maximum error for σ_x is reduced by a factor of 10 and the overall distribution of the predicted values is much more meaningful by adding the physical constraints. We conclude that adding simple physical constraints can significantly improve the prediction for the cases which are beyond the training regime under the same circumstances (e.g. number of epochs and network architecture). For the displacement fields, we did not observe a major improvement by adding the physical laws since the predictions of both approaches were quite good already.

3.4. Extension to 3D problems

We extended the formulation to cover the 3D case according to linear elastic assumptions (Eqs. (3) to (5)). A volume element is now considered according to Fig. 20 where the left surface is completely fixed and the right one is under $u_x = 0.05$ μm. All other surfaces are traction free. The results of the deformed configuration are now reported in Fig. 20 for the mixed PINNs formulation as well as the classical FE method. Further detailed comparisons are also provided in Fig. 21. One should mention that the training time for the 3D simulations increased drastically as the number of physical loss functions and collocation points also increased. Since we have 4 new functionals, the number of trainable parameters is also increased. We have chosen again 5 hidden layers with 40 neurons in each layer, and we used *tanh* as an activation function for all neurons. We train the network for 100 000 epochs with a full batch training. The number of collocation points is 9261. We choose the initial learning rate to be 0.001.

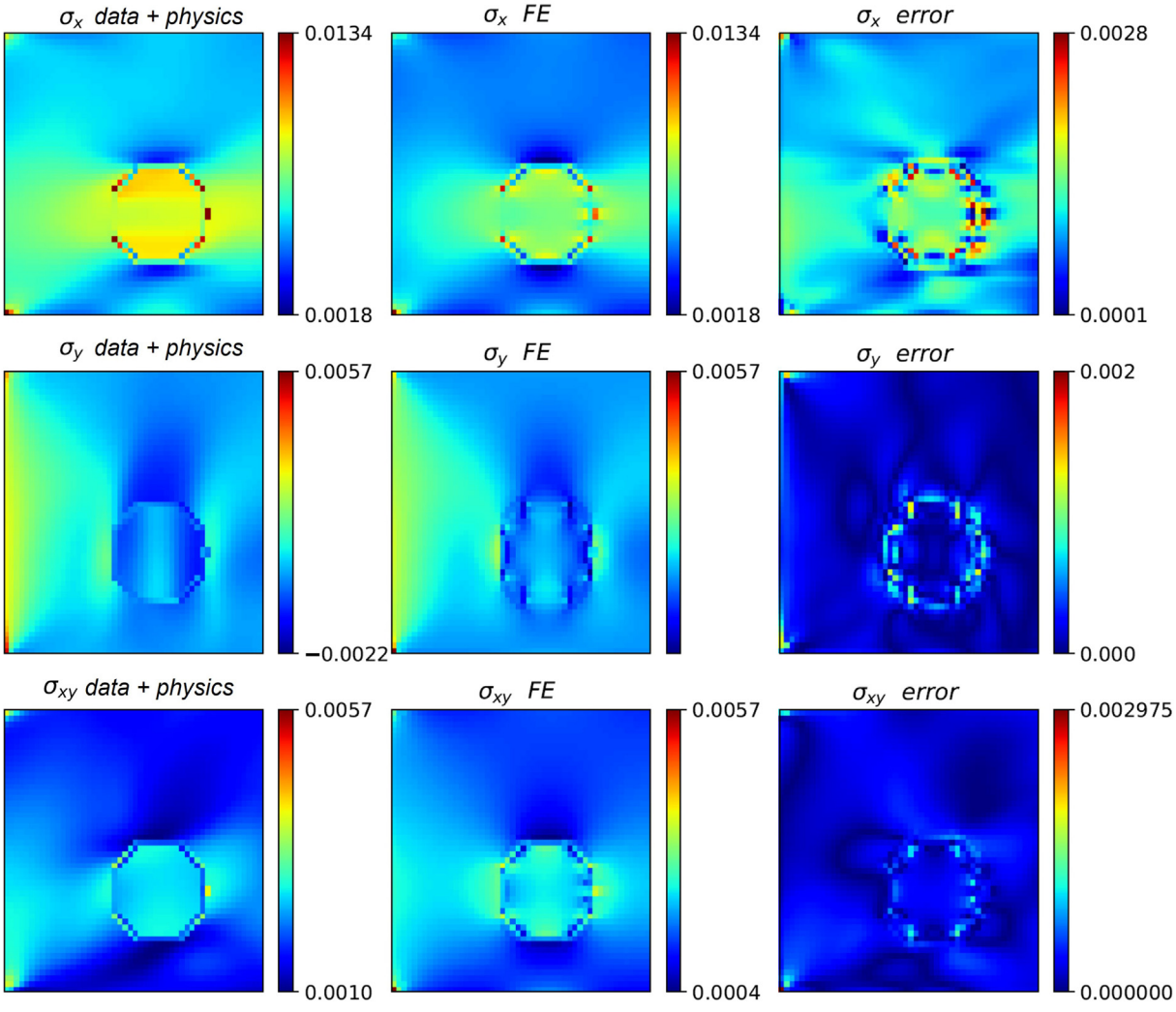

Fig. 18. Prediction of the trained physics-data-driven network ($\omega = 0.1$) for the unseen situation ($E_r \approx 0.083$). The network is trained based on physics and data using geometry 1 and different ratios of Young's modulus.

The components of the displacement vector are in a good agreement with the FE results. The source of difference for the stress components can be related to the limited number of epochs and collocation points we trained for. Therefore, in future developments, a combination of data and physics for the realistic micro-structure is very essential.

3.5. Diffusion problem

For the proposed mixed formulation based PINNs in this case, the output variables are the temperature field T as well as components of the heat flux vector $\boldsymbol{q}$. These are approximated via fully connected feed-forward neural networks (FFNN) just like the description provided in Section 3.1.

Similar to before, by denoting each neural network system by $\mathcal{N}$, we write the following solution components for the problem. Note that the proposed network is comparable with model E in Table 2 where in addition to the primary variable (T), its spatial derivative ($\boldsymbol{q}^O$) is also set as a network's output with a separated network for each quantity.

$$T = \mathcal{N}_T(\boldsymbol{X}; \boldsymbol{\theta}), \tag{35}$$

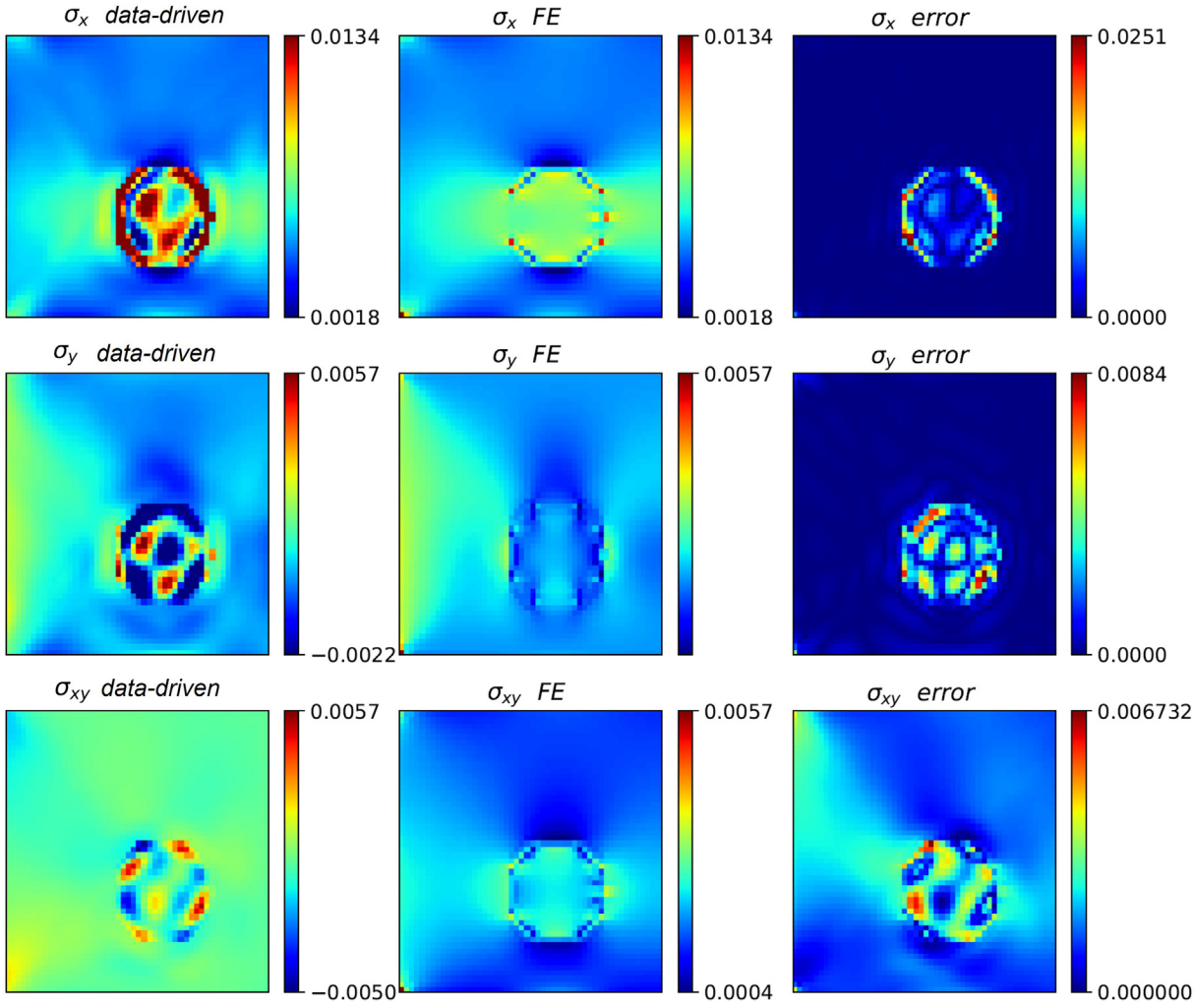

Fig. 19. Prediction of the trained data-driven network ($\omega = 0.0$) for the unseen situation. The network is trained based on data using geometry 1 and different ratios for Young's modulus.

$$q_x^O = \mathcal{N}_{q_x}(\boldsymbol{X};\boldsymbol{\theta}), \tag{36}$$

$$q_y^O = \mathcal{N}_{q_y}(\boldsymbol{X};\boldsymbol{\theta}). \tag{37}$$

Based on the described equations for the thermal problem (Eqs. (11) to (16)), the network architecture is shown in Fig. 22. For each network $\mathcal{N}$, the parameters are according to Table 3.

The total loss function and its components are defined below (see Eq. (38) and what follows). The network consists of three main parts. First, we have the loss terms applied to the temperature T. These terms are based on the energy form of the problem as well as the Dirichlet BCs ($\mathcal{L}_{EF}$ and $\mathcal{L}_{DBC}$). Next, we add the loss terms applied to the flux vector $\boldsymbol{q}$ as well as the Neumann BCs ($\mathcal{L}_{SF}$ and $\mathcal{L}_{NBC}$). Finally, we have the connection loss term $\mathcal{L}_{cnc}$. See also Appendix C, where we further exploited these terms according to the described BVP in Fig. 3.

Different loss terms are minimized with respect to the free parameters of the network. Again, we start with geometry 1 described in Fig. 2. The boundary conditions for the thermal problem are according to the right-hand side of Fig. 3. The position of collocation points for this problem is similar to the mechanical problem and is presented in Fig. 5. A finite element simulation is also performed based on regular quadrilateral meshes shown in

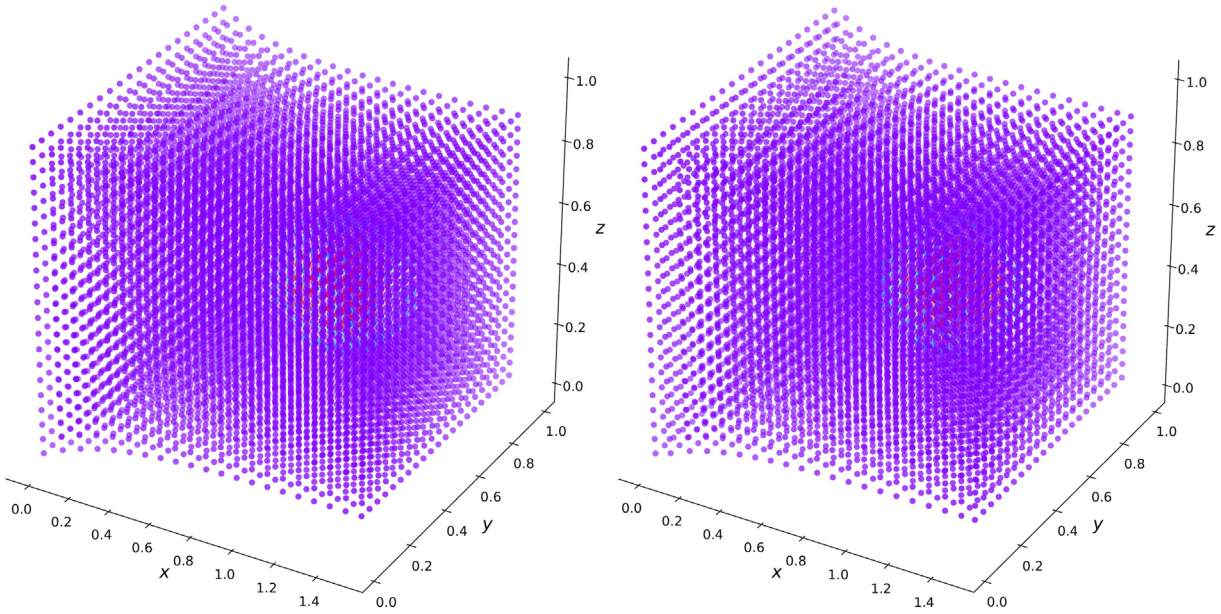

Fig. 20. Obtained results for the deformation of a 3D heterogeneous volume element from the introduced mixed formulation for PINNs (left) and FE calculations (right). The results for FE are shown in the middle of each element by means its center point. The results are magnified by a factor of 10.

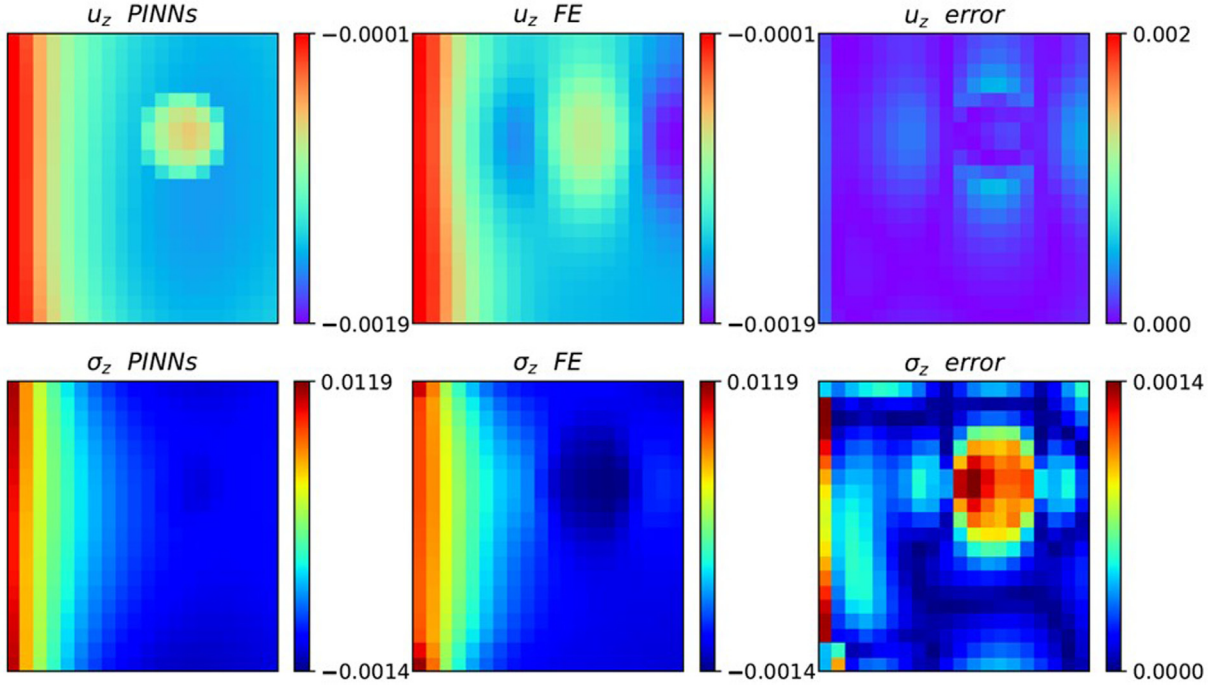

Fig. 21. Obtained results for the mechanical problem in 3D using the mixed formulation based PINNs and FEM. The right surface is subjected to prescribed displacement $\bar{u}_x = 0.05$ μm while the surface on the back is completely fixed. Other outer surfaces remain traction free.

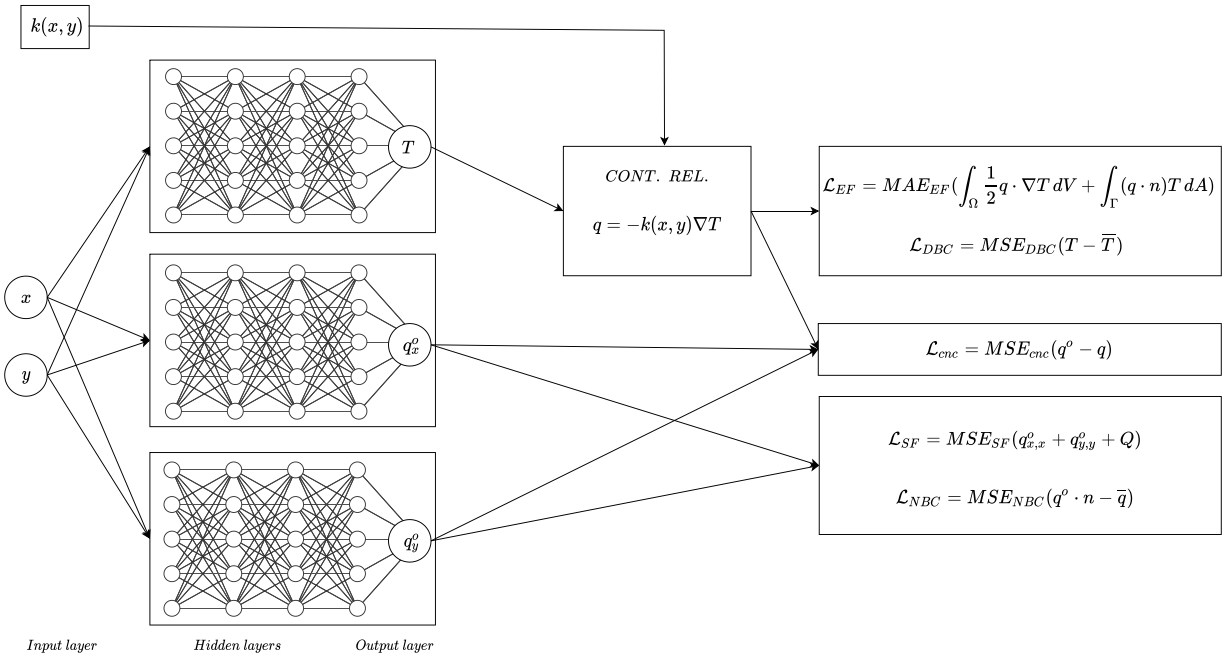

Fig. 22. Architecture and loss functions for the mixed formulation based network in the thermal problem.

Fig. 5. For the FE calculations, standard Q1 elements with bilinear shape functions are utilized [56].

$$\mathcal{L} = \underbrace{\mathcal{L}_{EF} + \mathcal{L}_{DBC}}_{\text{based on } T} + \mathcal{L}_{cnc} + \underbrace{\mathcal{L}_{SF} + \mathcal{L}_{NBC}}_{\text{based on } \boldsymbol{q}^o}, \tag{38}$$

$$\mathcal{L}_{EF} = \text{MAE}_{EF}\left(\int_\Omega \frac{1}{2}\boldsymbol{q}^T \cdot \nabla T \, dV + \int_\Gamma (\boldsymbol{q} \cdot \boldsymbol{n}) \, T \, dA\right), \tag{39}$$

$$\mathcal{L}_{DBC} = \text{MSE}_{DBC}\left(T - \overline{T}\right), \tag{40}$$

$$\mathcal{L}_{cnc} = \text{MSE}_{cnc}\left(\boldsymbol{q}^o - \boldsymbol{q}\right), \tag{41}$$

$$\mathcal{L}_{SF} = \text{MSE}_{SF}\left(\text{div}(\boldsymbol{q}^o)\right), \tag{42}$$

$$\mathcal{L}_{NBC} = \text{MSE}_{NBC}\left(\boldsymbol{q}^o \cdot \boldsymbol{n} - \overline{\boldsymbol{q}}\right). \tag{43}$$

In the above relation, the definition for the mean square error function is according to Eqs. (31) and (32).

In Fig. 23, different loss terms related to the governing equation, its energy form as well as the one related to BCs are plotted versus a number of epochs (iterations for training). We minimized the overall loss function for the same problem for five different times to make sure that the introduced network is always able to achieve a unique solution. Although not reported but we have observed similar same patterns as in Fig. 23 by repeating the minimization problem under same conditions.

The obtained results from PINNs and FEM are reported in Fig. 24 wherein the first row is the predicted temperature field and in the last two, the components of the heat flux vector are presented. In the first, second, and third columns, the results from PINNs, FE, and the difference between these two methodologies are presented, respectively. For all the output values, we observe a close agreement between the network predictions and those from the FE method. The difference is bold in the area where we have the sharp phase transition (border of heterogeneity). The maximum relative difference is about 12% and 9.6% for temperature and flux values, respectively. The average relative difference value is about 0.5% and 0.6% for temperature and flux values, respectively. Moreover, the results for predicted T and q_x along the cross-sections $x = 0.5$ μm are shown in Fig. 25.

The mixed formulation based network for the thermal problem is applied to examine more challenging cases. For this purpose, we use geometry 2, where we have a random complex heterogeneity. The collocation points are

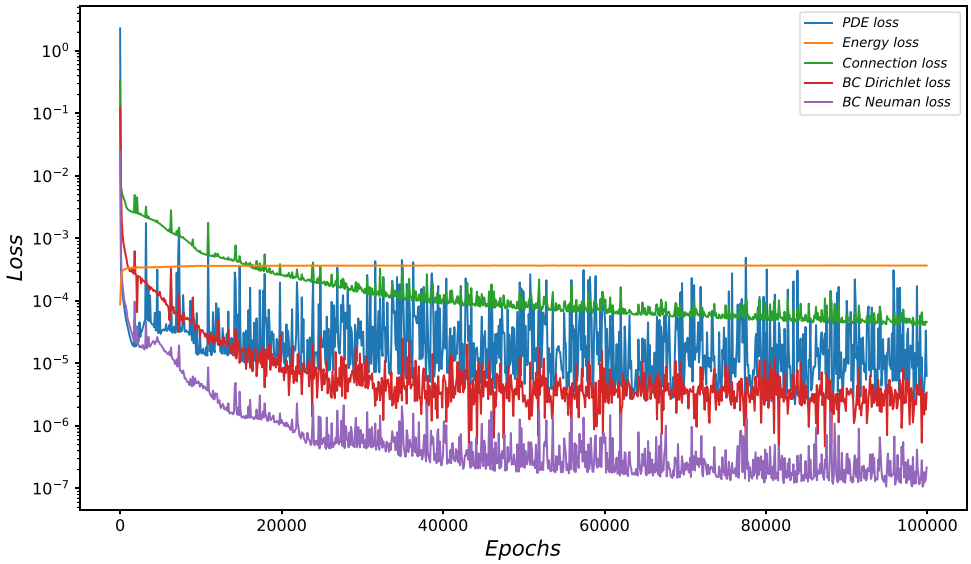

Fig. 23. Individual loss terms versus number of epoch for geometry 1.

shown on the left-hand side of Fig. 11. The middle part of the figure depicts the points where the trained network is evaluated. Note that the network is evaluated in a much denser grid (100×100 points). In other words, many of these points are not located in the set for minimization of the loss function (i.e., collocation points). The FE approach is also performed on a fine mesh (100×100 elements) shown on the right of Fig. 11. The results of the study are summarized in Fig. 26. Despite the complex shape of heterogeneity, the network can predict the distribution of deformation and stress components. The main difference is localized at the specific places in phase boundaries.

One possible remedy to reduce this error is to consider more collocation points (see Appendix B). As shown on the left-hand side of Fig. 27, we add about 800 collocation points in the vicinity of the interphase region where the highest error is observed. Loss functions of the same network are minimized for the new set of collocation points. According to Fig. 27, we observe a significant reduction in the difference between the mixed formulation based PINN and FE results using more collocation points.

4. Conclusion and outlooks

The potential of physics-informed neural networks for finding the solution to a given boundary value problem in a heterogeneous domain is studied. Heterogeneity is an important aspect related to the material microstructure. It offers a lot of flexibility for material design, and therefore accurate and fast prediction of material behavior at the microscale is essential. By combining several well-known ideas from the literature on PINN besides FE technology, we extend the available PINN models to a mixed formulation based version. Here the idea is to introduce the spatial gradient of the primary variable as an additional output for the network. Furthermore, a separated network is addressed for each output. Then, the output quantities are connected and constrained based on the governing equation (PDE) and its energy form (obtained from the weak form). Two problems governed by mechanical equilibrium plus a steady-state diffusion equation are investigated. The results from DL are compared against the solution of the standard FEM. Early studies show that the proposed approach predicts the solution without having any initial data. The current work also summarizes overriding knowledge for people in computational science to utilize DL methods in their studies.

Despite many valuable contributions [16–19], further studies are needed to fully understand the role of the networks (hyper) parameter and their relation to the final obtained solution (e.g., layer depth, learning rate, location, and distribution of collocation points, etc.). Moreover, one of the main advantages of PINNs is their simple implementation. Integrating new boundary conditions and complex geometries or investigating new governing

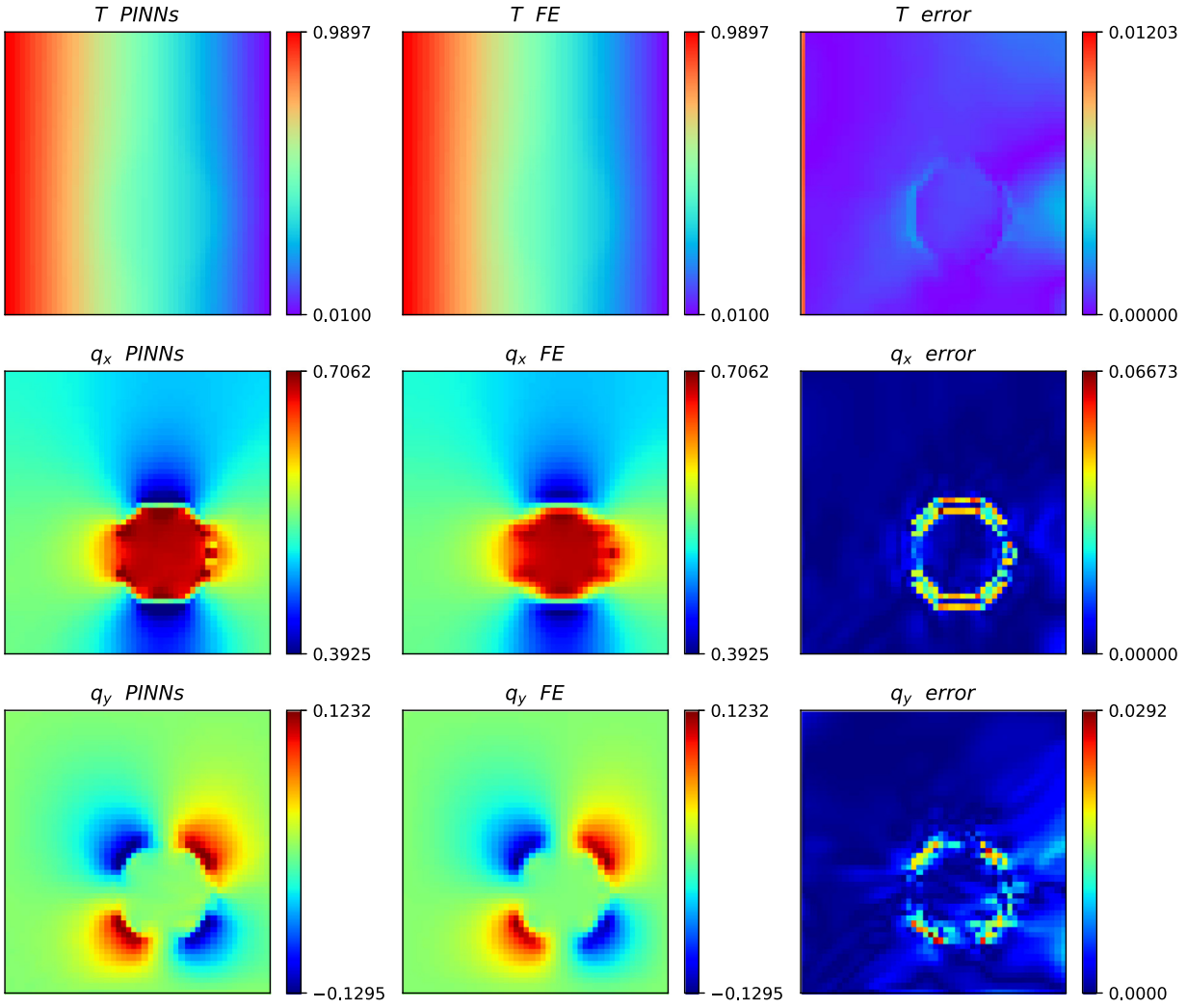

Fig. 24. Results for the thermal problem using the mixed formulation based PINNs and the FEM.

equations are straight forward. The main drawback for PINNs at the current moment is the computational time. For instance, for the problems discussed in this work one needs in order of few hours to minimize the loss function and find the solution, whereas the same problem took less than a few minutes using FEM (upon having the developed codes and models). Certainly, the optimizer or design of the network for DL should be improved to speed up this process. As examples of developments in this direction, one can mention utilizing other minimizers based on Broyden–Fletcher–Goldfarb–Shanno (BFGS) methods [19], enhancing the loss functions with additional gradient terms [57] or breaking the given domain to sub-domains and training a separate network for each one [37].

One should note that the DL methods can be trained for different BVPs and various loading conditions and material properties (i.e., all the interesting parameters one requires in each engineering task). For example the distribution of the stiffness matrix or different type of boundary conditions can be an additional input for the network. As a result, the computational cost of these methods will decay as they get trained by realizing the new systems. See schematic representation of this latter discussion in Fig. 28. The last point, in the long-term, has the potential to solve two main issues from computational science: computational cost and accuracy simultaneously. Furthermore, the available data from different computational strategies in various domains shall not be wasted and can be used (recycled) for training the ultimate network. Please note that there is also another strategy where we feed the network with a sufficient amount of data or situation (case studies) at once and then train the network. See

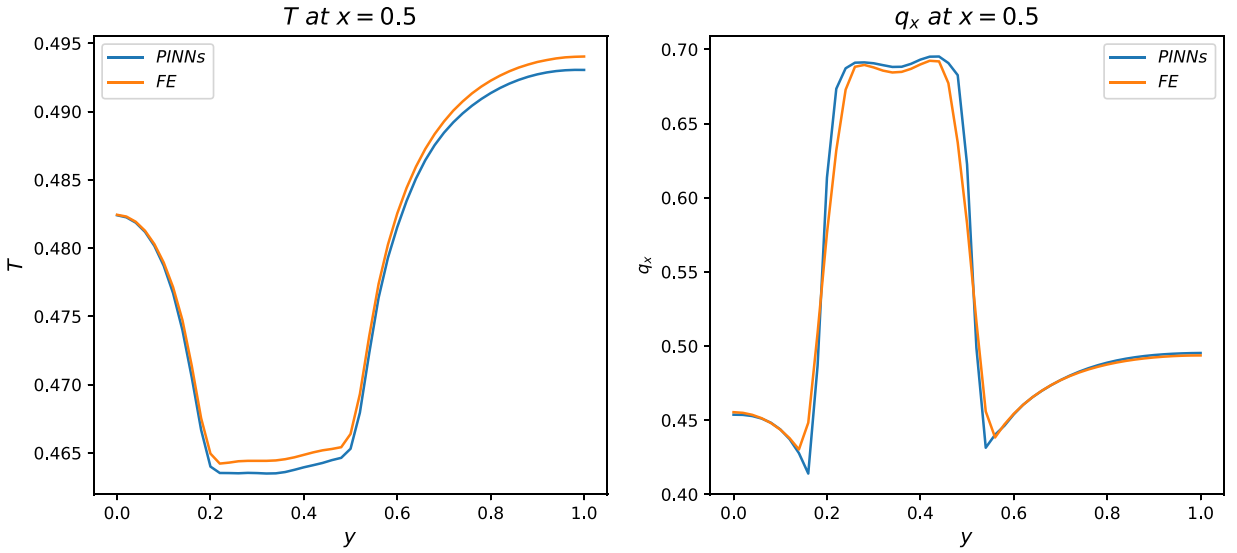

Fig. 25. Comparison between the mixed formulation based PINNs and FE along $x = 0.5$ μm direction.

for example [6,8,58–60]. What we present here is one simple way of combination of data and physics. In the future studies, we should study different ways one can enhance the prediction of data-driven approaches with physical constrains.

This work focused on isotropic elastic material in a 2D setting. Therefore in future developments, one should investigate this further for geometrical and material nonlinear settings in 3D. Moreover, it is interesting to explore the idea of solving multiphysics problems where we also have system evolution in time.

CRediT authorship contribution statement

Shahed Rezaei: Conceptualization, Methodology, Supervision, Writing – review & editing. **Ali Harandi:** Methodology, Software, Writing – review & editing. **Ahmad Moeineddin:** Methodology, Software, Writing – review & editing. **Bai-Xiang Xu:** Supervision, Review & editing. **Stefanie Reese:** Supervision, Funding acquisition, Review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

The authors gratefully acknowledge the computing time granted on high performance computers of RWTH-Aachen university.

Appendix A. Ideas to enhance the learning rate based on the Newton–Raphson method

For the training, we intend to minimize the total loss function $\mathcal{L}$ with respect to the network parameter $\boldsymbol{\theta}$ (i.e. $\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{X};\boldsymbol{\theta})$). As discussed in Section 3.1, through an iterative process, one can reach the (global) minimum point using gradient decent method:

$$\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_i). \tag{44}$$

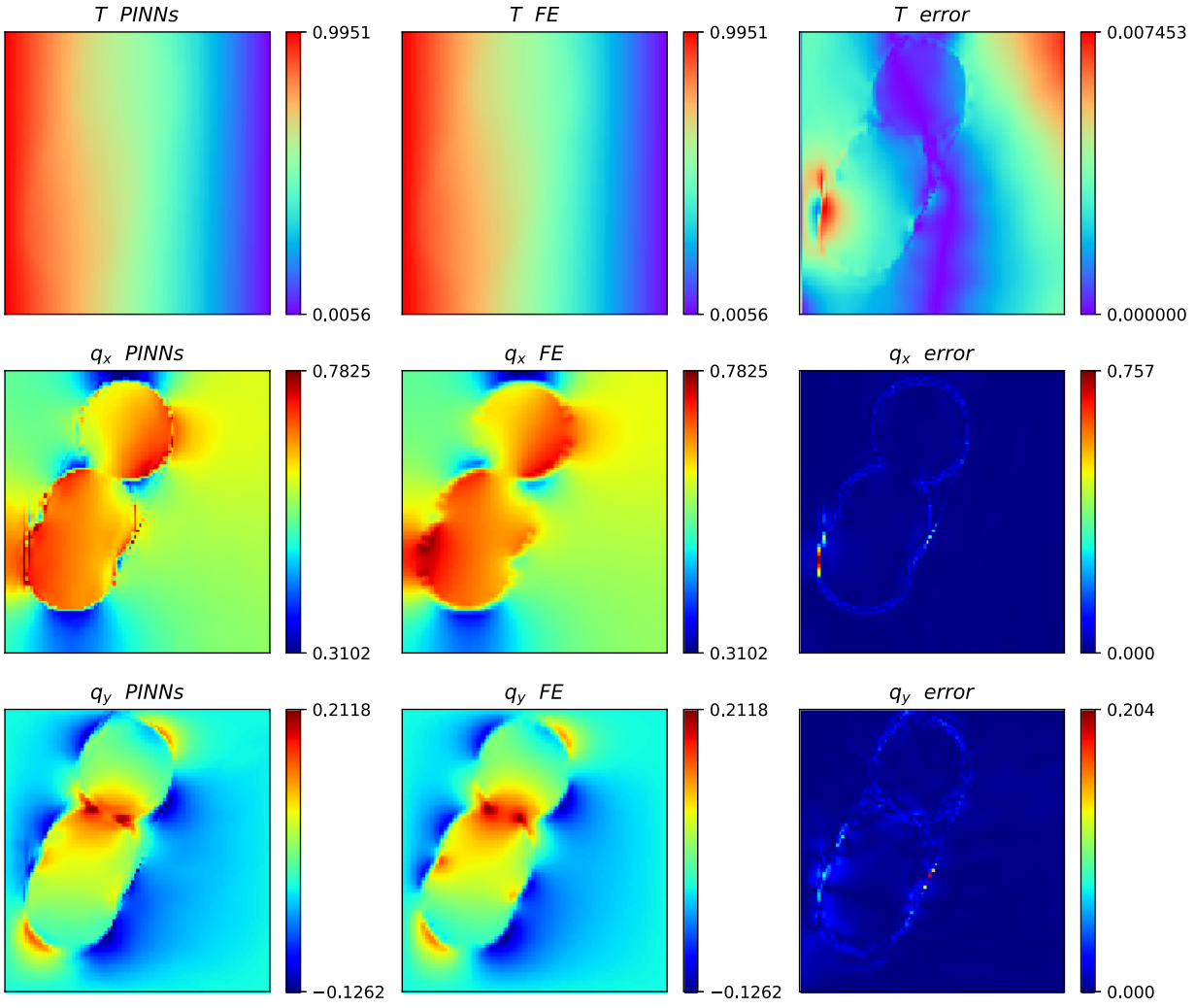

Fig. 26. Obtained results from the mixed formulation based PINNs and the FEM for the thermal problem.

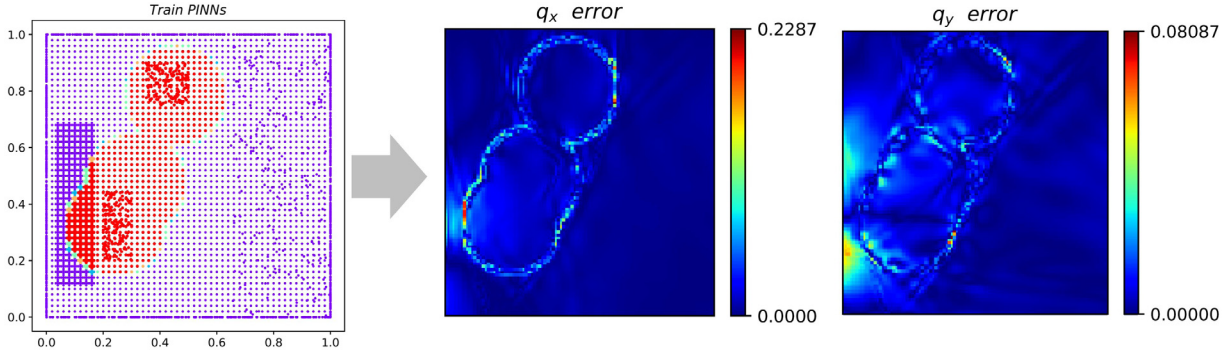

Fig. 27. Difference between the mixed formulation based PINN and FE after refinement of collocation points at the interface region. The results obtained from PINN are (by two orders for q_y) improved after minimizing the network using the new set of collocation points shown on the left-hand side.

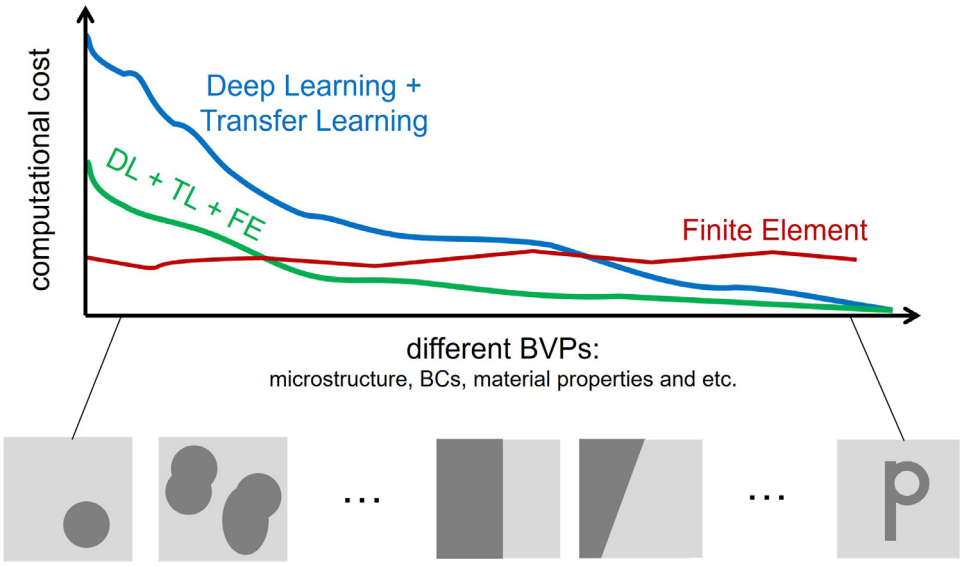

Fig. 28. A comparison between available solver such as finite element (FE) and upcoming methodologies such as deep learning (DL). At the moment the computational cost of DL is higher than FE. This cost can go to zero by combining the two methods as well as utilizing transfer learning (TL) in the future developments.

On the other hand, at the minimum point one can argue that we have

$$\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}^*) = \left.\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}\right|_{\boldsymbol{\theta}^*} = \mathbf{0} = \boldsymbol{L}(\boldsymbol{\theta}^*). \tag{45}$$

In other words, the minimization process can be interpreted as a root-finding problem and the solution is where $\boldsymbol{L}$ goes towards zero. For this problem, we apply the standard Newton–Raphson method. Therefore, one can write

$$\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i - \left.\left(\frac{\partial \boldsymbol{L}}{\partial \boldsymbol{\theta}}\right)^{-1}\right|_{\boldsymbol{\theta}_i} \boldsymbol{L}(\boldsymbol{\theta}_i). \tag{46}$$

According to our definition we have $\boldsymbol{L}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta})$. Substituting the latter expression into Eq. (46), we conclude that

$$\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i - \underbrace{\left.\left(\frac{\partial^2 \mathcal{L}}{\partial \boldsymbol{\theta}^2}\right)^{-1}\right|_{\boldsymbol{\theta}_i}}_{\boldsymbol{\alpha}_i} \nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}_i). \tag{47}$$

Comparing Eqs. (44) and (47), one can conclude that for a faster convergence to the minimum point, the learning rate can take a tonsorial form $\boldsymbol{\alpha}$. The latter suggests that for going in direction of the minimum point, one can consider some weightings to different components of the vector $\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta}_i)$ (instead of a scalar number). On the other hand, the calculation of the second derivative of the loss function and its inversion might not be trivial and gets costly. Therefore, a proper approximation of this tensor quantity can be extremely beneficial for future studies. See also other algorithms such as BFGS methods for optimization and training [19].

Appendix B. Influence of the derivation order, number of collocation points, and number of epochs on the obtained solution

We consider the following function $y(x)$ as the desired solution:

$$y(x) = \sin(x) + 0.1\ x + 0.1. \tag{48}$$

To construct our differential equations, we take the first and the second derivative of the above relation concerning the parameter x. Therefore, we write the following 1st and 2nd order ordinary differential equations (ODEs)

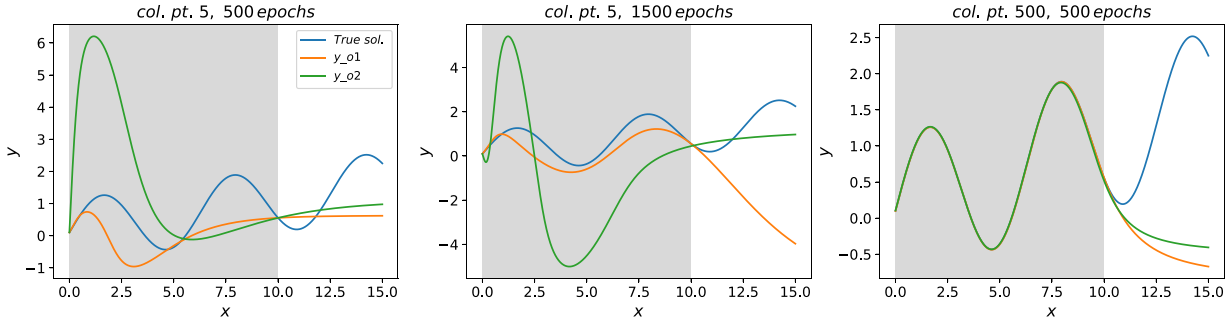

Fig. 29. Comparison between the solution obtained from first and second order ODEs.

subjected to the following boundary conditions:

$$y' = \frac{\mathrm{d}y}{\mathrm{d}x} = \cos(x) + 0.1, \quad y(0) = 0.1, \quad y(10) = 0.556, \tag{49}$$

$$y'' = \frac{\mathrm{d}^2 y}{\mathrm{d}x^2} = -\sin(x), \quad y(0) = 0.1, \quad y(10) = 0.556. \tag{50}$$

The idea is to obtain (reproduce) the function $y(x)$, by feeding the above-mentioned ODEs to two separate neural networks with the same architecture and hyper parameters. As a result, one can study the influence of derivation order on the performance of the deep learning algorithm which is used to find the solution $y(x)$. Therefore, we define the following loss functions based on the above ODEs and their relative BCs:

$$\mathcal{L}_{o1} = \mathrm{MSE}\left(\frac{\mathrm{d}y_{o1}}{\mathrm{d}x} - \cos(x) - 0.1\right) + \mathrm{MSE}\left(y_{o1}(0) - 0.1\right) + \mathrm{MSE}\left(y_{o1}(10) - 0.556\right), \tag{51}$$

$$\mathcal{L}_{o2} = \mathrm{MSE}\left(\frac{\mathrm{d}^2 y_{o2}}{\mathrm{d}x^2} + \sin(x)\right) + \mathrm{MSE}\left(y_{o2}(0) - 0.1\right) + \mathrm{MSE}\left(y_{o2}(10) - 0.556\right). \tag{52}$$

For having a fair comparison, the above loss functions are minimized by utilizing the same network, which has 4 layers and 20 neurons in each layer. Moreover, we used *tanh* as an activation function for all the neurons, and we kept the learning rate as 0.001 in the Adam optimizer. In the below figure, results obtained from these two networks (y_{o1} and y_{o2}) as well the as the expected solution ($y_{true}(x) = y(x)$) are plotted for a different number of collocation points and epochs. The gray area represents the training zone where we have distributed the collocation points equally distanced from 0 to 10. The obtained solutions are plotted further (up to 15) to show the extrapolation behavior. On the left-hand side, we observe a poor prediction since we used very few collocation points and a rather low number of iterations (epochs). The predictions improved by adding more collocation points and/or increasing the number of epochs. This study clearly shows that by having a lower degree of derivation, the predictions become more accurate (compare orange and blue curves). If we increase the number of collocation points and epochs, the results of two given ODEs converge to the exact solution (see Fig. 29).

To demonstrate the influence of the number of collocation points, we compare the convergence of both forms of the first and second-order ODEs based on the different number of collocation points. The collocation points are changed from 10 to 100 and we kept the number of epochs at 1500 for all the calculations. Since the initial values of trainable parameters could affect the final solution, we repeated the training three times. After each training, we calculated the mean of the absolute difference between the predicted values to the true solutions. The results are reported in Fig. 30. First, we observe that by increasing the number of collocation points, the solution from both types of ODEs convergences. Second, the MAE of first-order ODEs is always lower than second-order ODEs. It is worth to mention that for the 1st order ODE, one needs only one boundary condition while we provide the network with two (see Eq. (51)). This point might be another explanation for lower errors in the solution of the 1st order ODE. Finally, The randomness or stability of both training is reduced by using more collocation points. After a certain number of collocation points, the standard deviation of the prediction in first-order ODEs is negligible.

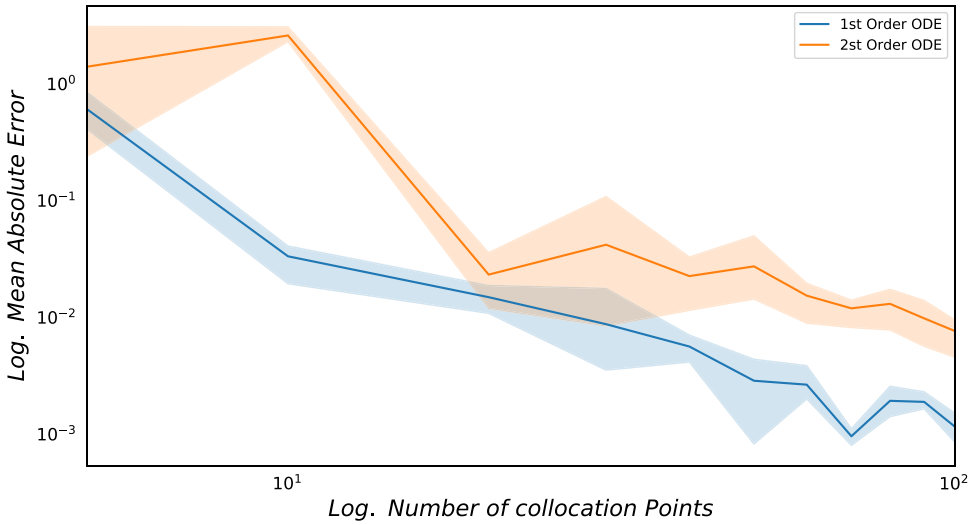

Fig. 30. MAE by increasing the number of collocation points for solution of the 1st and 2nd order ODE.

Appendix C. Details on the calculation of the loss functions

For the BVP in Fig. 3, we further expand the loss functions introduced in Eqs. (25)–(30). First, components of the strain tensor are computed based on the kinematics relation and by utilizing the components of the deformation vector from the network output:

$$\mathcal{N}_{\varepsilon_x}(\boldsymbol{X};\boldsymbol{\theta}) = \partial_x \mathcal{N}_{u_x}, \tag{53}$$

$$\mathcal{N}_{\varepsilon_y}(\boldsymbol{X};\boldsymbol{\theta}) = \partial_y \mathcal{N}_{u_y}, \tag{54}$$

$$\mathcal{N}_{\varepsilon_{xy}}(\boldsymbol{X};\boldsymbol{\theta}) = \frac{1}{2}\left(\partial_y \mathcal{N}_{u_x} + \partial_x \mathcal{N}_{u_y}\right). \tag{55}$$

Next, by employing the constitutive relations we have the components of the stress tensor as

$$\begin{aligned}
\mathcal{N}_{\sigma_x}(\boldsymbol{X};\boldsymbol{\theta}) &= \frac{E(\boldsymbol{X})}{(1+\nu(\boldsymbol{X}))(1-2\nu(\boldsymbol{X}))}\left((1-\nu(\boldsymbol{X}))\mathcal{N}_{\varepsilon_x} + \nu(\boldsymbol{X})\mathcal{N}_{\varepsilon_y}\right),\\
\mathcal{N}_{\sigma_y}(\boldsymbol{X};\boldsymbol{\theta}) &= \frac{E(\boldsymbol{X})}{(1+\nu(\boldsymbol{X}))(1-2\nu(\boldsymbol{X}))}\left(\nu(\boldsymbol{X})\mathcal{N}_{\varepsilon_x} + (1-\nu(\boldsymbol{X}))\mathcal{N}_{\varepsilon_y}\right),\\
\mathcal{N}_{\sigma_{xy}}(\boldsymbol{X};\boldsymbol{\theta}) &= \frac{E(\boldsymbol{X})}{2(1+\nu(\boldsymbol{X}))}\mathcal{N}_{\varepsilon_{xy}}.
\end{aligned} \tag{56}$$

Please note that, for simplicity, the dependency of neural network $\mathcal{N}(\boldsymbol{X};\boldsymbol{\theta})$ on $\boldsymbol{X}$ (input parameters), and $\boldsymbol{\theta}$ (network trainable parameters) is not shown on the right-hand side of the above equations and in the loss terms. The similar trend is utilized also for the thermal part. Finally, the loss functions are further expanded as below

$$\mathcal{L}_{EF} = \left| -\frac{1}{2N_b}\sum_{\boldsymbol{X}\in\Omega}\left(\mathcal{N}_{\sigma_x}\mathcal{N}_{\varepsilon_x} + \mathcal{N}_{\sigma_y}\mathcal{N}_{\varepsilon_y} + \mathcal{N}_{\sigma_{xy}}\mathcal{N}_{\varepsilon_{xy}}\right) + \frac{1}{N^R_{NBC}}\sum_{\boldsymbol{X}\in\Gamma^R_N}\left(\mathcal{N}_{\sigma_x}\mathcal{N}_{u_x}\right)\right|, \tag{57}$$

$$\mathcal{L}_{DBC} = \frac{1}{N^L_{DBC}}\sum_{\boldsymbol{X}\in\Gamma^L_D}\left(\left(\mathcal{N}_{u_x}-0\right)^2 + \left(\mathcal{N}_{u_y}-0\right)^2\right) + \frac{1}{N^R_{DBC}}\sum_{\boldsymbol{X}\in\Gamma^R_D}\left(\mathcal{N}_{u_x}-0.05\right)^2, \tag{58}$$

$$\mathcal{L}_{cnc} = \frac{1}{N_b}\sum_{\boldsymbol{X}\in\Omega}\left(\left(\mathcal{N}_{\sigma^o_x}-\mathcal{N}_{\sigma_x}\right)^2 + \left(\mathcal{N}_{\sigma^o_y}-\mathcal{N}_{\sigma_y}\right)^2 + \left(\mathcal{N}_{\sigma^o_{xy}}-\mathcal{N}_{\sigma_{xy}}\right)^2\right), \tag{59}$$

$$\mathcal{L}_{SF} = \frac{1}{N_b}\sum_{\boldsymbol{X}\in\Omega}\left(\left(\partial_x\mathcal{N}_{\sigma^o_x} + \partial_y\mathcal{N}_{\sigma^o_{xy}}\right)^2 + \left(\partial_y\mathcal{N}_{\sigma^o_y} + \partial_x\mathcal{N}_{\sigma^o_{xy}}\right)^2\right), \tag{60}$$

$$\mathcal{L}_{NBC} = \frac{1}{N^T_{NBC}} \sum_{\boldsymbol{X}\in\Gamma^T_N} \left(\left(\mathcal{N}_{\sigma^o_y} - 0\right)^2 + \left(\mathcal{N}_{\sigma^o_{xy}} - 0\right)^2 \right) \tag{61}$$

$$+ \frac{1}{N^B_{NBC}} \sum_{\boldsymbol{X}\in\Gamma^B_N} \left(\left(\mathcal{N}_{\sigma^o_y} - 0\right)^2 + \left(\mathcal{N}_{\sigma^o_{xy}} - 0\right)^2 \right) \tag{62}$$

$$+ \frac{1}{N^R_{NBC}} \sum_{\boldsymbol{X}\in\Gamma^R_N} \left(\mathcal{N}_{\sigma^o_{xy}} - 0\right)^2 . \tag{63}$$

In the above relations Γ^L, Γ^R, Γ^T and Γ^B denote the boundaries (points) at left, right, top and bottom boundary. The sub-indexes *DBC* and *NBC* stand for Dirichlet and Neumann boundary conditions.

Note that in the above relations, we simply omit the terms which are zero. For example, having traction-free boundaries on top and bottom and having fixed displacement on the left edge resulted in further simplifications.

For the BVP in Fig. 3, we further expand the loss functions introduced in Eqs. (38)–(43). First, components of the temperature gradient vector are computed based on the temperature value from the network output:

$$\mathcal{N}_{\nabla_x T}(\boldsymbol{X};\boldsymbol{\theta}) = \partial_x \mathcal{N}_T , \tag{64}$$

$$\mathcal{N}_{\nabla_y T}(\boldsymbol{X};\boldsymbol{\theta}) = \partial_y \mathcal{N}_T . \tag{65}$$

Next, we compute the components of the flux tensor according to

$$\mathcal{N}_{q_x}(\boldsymbol{X};\boldsymbol{\theta}) = -k(\boldsymbol{X})\mathcal{N}_{\nabla_x T} , \tag{66}$$

$$\mathcal{N}_{q_y}(\boldsymbol{X};\boldsymbol{\theta}) = -k(\boldsymbol{X})\mathcal{N}_{\nabla_y T} . \tag{67}$$

Finally, the loss functions are further expanded as below

$$\mathcal{L}_{EF} = \left| \frac{1}{2N_b} \sum_{\boldsymbol{X}\in\Omega} \left(\mathcal{N}_{q_x}\mathcal{N}_{\nabla_x T} + \mathcal{N}_{q_y}\mathcal{N}_{\nabla_y T}\right) + \frac{1}{N^L_{NBC}} \sum_{\boldsymbol{X}\in\Gamma^L_N} \left(\mathcal{N}_{q_x}\mathcal{N}_T\right) \right| , \tag{68}$$

$$\mathcal{L}_{DBC} = \frac{1}{N^L_{DBC}} \sum_{\boldsymbol{X}\in\Gamma^L_D} \left(\mathcal{N}_T - 1\right)^2 + \frac{1}{N^R_{DBC}} \sum_{\boldsymbol{X}\in\Gamma^R_D} \left(\mathcal{N}_T - 0\right)^2 , \tag{69}$$

$$\mathcal{L}_{cnc} = \frac{1}{N_b} \sum_{\boldsymbol{X}\in\Omega} \left(\left(\mathcal{N}_{q^o_x} - \mathcal{N}_{q_x}\right)^2 + \left(\mathcal{N}_{q^o_y} - \mathcal{N}_{q_y}\right)^2 \right) , \tag{70}$$

$$\mathcal{L}_{SF} = \frac{1}{N_b} \sum_{\boldsymbol{X}\in\Omega} \left(\partial_x \mathcal{N}_{q^o_x} + \partial_y \mathcal{N}_{q^o_y}\right)^2 , \tag{71}$$

$$\mathcal{L}_{NBC} = \sum_{\boldsymbol{X}\in\Gamma^T_N} \frac{1}{N^T_{NBC}} \left(\mathcal{N}_{q^o_y} - 0\right)^2 \tag{72}$$

$$+ \sum_{\boldsymbol{X}\in\Gamma^B_N} \frac{1}{N^B_{NBC}} \left(\mathcal{N}_{q^o_y} - 0\right)^2 \tag{73}$$

It is worth mentioning that the external work in Eq. (68) vanishes due to having $T = 0$ on the right-hand side of the boundary value problem. Calculation of the integral terms is performed based on simple summation over organized grid points. Readers are also encouraged to see [17] for other possible approaches.

References

[1] Frederic E. Bock, Roland C. Aydin, Christian J. Cyron, Norbert Huber, Surya R. Kalidindi, Benjamin Klusemann, A review of the application of machine learning and data mining approaches in continuum materials mechanics, Front. Mater. 6 (2019).

[2] Grace C.Y. Peng, Mark Alber, Adrian Buganza Tepole, William R. Cannon, Suvranu De, Savador Dura-Bernal, Krishna Garikipati, George Karniadakis, William W. Lytton, Paris Perdikaris, Linda Petzold, Ellen Kuhl, Multiscale modeling meets machine learning: What can we learn? Arch. Comput. Methods Eng. (2021).

[3] Mauricio Fernández, Shahed Rezaei, Jaber Rezaei Mianroodi, Felix Fritzen, Stefanie Reese, Application of artificial neural networks for the prediction of interface mechanics: A study on grain boundary constitutive behavior, Adv. Model. Simul. Eng. Sci. 7 (1) (2020) 1–27.
[4] Minglang Yin, Enrui Zhang, Yue Yu, George Em Karniadakis, Interfacing finite elements with deep neural operators for fast multiscale modeling of mechanics problems, 2022.
[5] Yu-Chuan Hsu, Chi-Hua Yu, Markus J. Buehler, Using deep learning to predict fracture patterns in crystalline solids, Matter 3 (1) (2020) 197–211.
[6] Jaber Rezaei Mianroodi, Nima H. Siboni, Dierk Raabe, Teaching solid mechanics to artificial intelligence—A fast solver for heterogeneous materials, Npj Comput. Mater. 7 (1) (2021) 1–10.
[7] Anindya Bhaduri, Ashwini Gupta, Lori Graham-Brady, Stress field prediction in fiber-reinforced composite materials using a deep learning approach, 2021.
[8] Jaber Rezaei Mianroodi, Shahed Rezaei, Nima H Siboni, Bai-Xiang Xu, Dierk Raabe, Lossless multi-scale constitutive elastic relations with artificial intelligence, Npj Comput. Mater. 8 (1) (2022) 1–12.
[9] Binbin Lin, Yang Bai, Bai-Xiang Xu, Data-driven microstructure sensitivity study of fibrous paper materials, Mater. Des. 197 (2021) 109193.
[10] Siddhant Kumar, Stephanie Tan, Li Zheng, Dennis M. Kochmann, Inverse-designed spinodoid metamaterials, Npj Comput. Mater. 6 (2020) 110.
[11] M. Mozaffar, R. Bostanabad, W. Chen, K. Ehmann, J. Cao, M.A. Bessa, Deep learning predicts path-dependent plasticity, Proc. Natl. Acad. Sci. 116 (52) (2019) 26414–26420.
[12] Kun Wang, WaiChing Sun, A multiscale multi-permeability poroplasticity model linked by recursive homogenizations and deep learning, Comput. Methods Appl. Mech. Engrg. (ISSN: 0045-7825) 334 (2018) 337–380.
[13] Kevin Linka, Markus Hillgärtner, Kian P. Abdolazizi, Roland C. Aydin, Mikhail Itskov, Christian J. Cyron, Constitutive artificial neural networks: A fast and general approach to predictive data-driven constitutive modeling by deep learning, J. Comput. Phys. (ISSN: 0021-9991) 429 (2021) 110010.
[14] Filippo Masi, Ioannis Stefanou, Paolo Vannucci, Victor Maffi-Berthier, Thermodynamics-based artificial neural networks for constitutive modeling, J. Mech. Phys. Solids (ISSN: 0022-5096) 147 (2021) 104277.
[15] M. Fernández, M. Jamshidian, T. Böhlke, Kristian Kersting, Oliver Weeger, Anisotropic hyperelastic constitutive models for finite deformations combining material theory and data-driven approaches with application to cubic lattice metamaterials, Comput. Mech. 67 (2021) 653–677.
[16] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[17] Jan N. Fuhg, Nikolaos Bouklas, The mixed deep energy method for resolving concentration features in finite strain hyperelasticity, J. Comput. Phys. 451 (2022) 110839.
[18] Hongwei Guo, Xiaoying Zhuang, Pengwan Chen, Naif Alajlan, Timon Rabczuk, Analysis of three-dimensional potential problems in non-homogeneous media with physics-informed deep collocation method using material transfer learning and sensitivity analysis, Eng. Comput. (2022).
[19] Alexander Henkes, Henning Wessels, Rolf Mahnken, Physics informed neural networks for continuum micromechanics, Comput. Methods Appl. Mech. Engrg. 393 (2022) 114790.
[20] Hyuk Lee, In Seok Kang, Neural algorithm for solving differential equations, J. Comput. Phys. (ISSN: 0021-9991) 91 (1) (1990) 110–131.
[21] Dimitris C. Psichogios, Lyle H. Ungar, A hybrid neural network-first principles approach to process modeling, AIChE J. 38 (10) (1992) 1499–1511.
[22] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (5) (1998) 987–1000.
[23] Shengze Cai, Zhiping Mao, Zhicheng Wang, Minglang Yin, George Em Karniadakis, Physics-informed neural networks (PINNs) for fluid mechanics: A review, Acta Mech. Sinica (2022).
[24] George Em Karniadakis, Ioannis G Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, Liu Yang, Physics-informed machine learning, Nat. Rev. Phys. 3 (6) (2021) 422–440.
[25] Jens Berg, Kaj Nyström, A unified deep artificial neural network approach to partial differential equations in complex geometries, Neurocomputing 317 (2018) 28–41.
[26] Jan Blechschmidt, Oliver G. Ernst, Three ways to solve partial differential equations with neural networks — A review, GAMM-Mitt. 44 (2021).
[27] Navid Zobeiry, Keith D. Humfeld, A physics-informed machine learning approach for solving heat transfer equation in advanced manufacturing and engineering applications, Eng. Appl. Artif. Intell. 101 (2021) 104232.
[28] E. Samaniego, C. Anitescu, S. Goswami, V.M. Nguyen-Thanh, H. Guo, K. Hamdia, X. Zhuang, T. Rabczuk, An energy approach to the solution of partial differential equations in computational mechanics via machine learning: Concepts, implementation and applications, Comput. Methods Appl. Mech. Engrg. 362 (2020) 112790.
[29] Mengwu Guo, Ehsan Haghighat, An energy-based error bound of physics-informed neural network solutions in elasticity, 2020.
[30] Shengze Cai, Zhicheng Wang, Sifan Wang, Paris Perdikaris, George Em Karniadakis, Physics-informed neural networks for heat transfer problems, J. Heat Transfer 143 (6) (2021).
[31] Ehsan Haghighat, Maziar Raissi, Adrian Moure, Hector Gomez, Ruben Juanes, A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics, Comput. Methods Appl. Mech. Engrg. 379 (2021) 113741.

[32] Rajat Arora, Machine learning-accelerated computational solid mechanics: Application to linear elasticity, 2021.
[33] Enrui Zhang, Ming Dao, George Em Karniadakis, Subra Suresh, Analyses of internal structures and defects in materials using physics-informed neural networks, Sci. Adv. 8 (7) (2022) eabk0644.
[34] Diab W. Abueidda, Seid Koric, Rashid Abu Al-Rub, Corey M. Parrott, Kai A. James, Nahil A. Sobh, A deep learning energy method for hyperelasticity and viscoelasticity, 2022.
[35] Chengping Rao, Hao Sun, Yang Liu, Physics-informed deep learning for computational elastodynamics without labeled data, J. Eng. Mech. 147 (8) (2021) 04021043.
[36] Gaurav Kumar Yadav, Sundararajan Natarajan, Balaji Srinivasan, Distributed PINN for linear elasticity — A unified approach for smooth, singular, compressible and incompressible media, Int. J. Comput. Methods 2142008.
[37] Ameya D. Jagtap, Ehsan Kharazmi, George Em Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems, Comput. Methods Appl. Mech. Engrg. 365 (2020) 113028.
[38] Zhizhou Zhang, Grace X. Gu, Physics-informed deep learning for digital materials, Theor. Appl. Mech. Lett. 11 (1) (2021) 100220.
[39] Enrui Zhang, Minglang Yin, George Em Karniadakis, Physics-informed neural networks for nonhomogeneous material identification in elasticity imaging, 2020, arXiv, arXiv:2009.04525.
[40] Qizhi He, David Barajas-Solano, Guzel Tartakovsky, Alexandre Tartakovsky, Physics-informed neural networks for multiphysics data assimilation with application to subsurface transport, Adv. Water Resour. 141 (2020) 103610.
[41] Long Nguyen, Maziar Raissi, Padmanabhan Seshaiyer, Efficient physics informed neural networks coupled with domain decomposition methods for solving coupled multi-physics problems, in: Advances in Computational Modeling and Simulation, Springer, 2022, pp. 41–53.
[42] Ravi G. Patel, Indu Manickam, Nathaniel A. Trask, Mitchell A. Wood, Myoungkyu Lee, Ignacio Tomas, Eric C. Cyr, Thermodynamically consistent physics-informed neural networks for hyperbolic systems, J. Comput. Phys. 449 (2022) 110754.
[43] Somdatta Goswami, Cosmin Anitescu, Souvik Chakraborty, Timon Rabczuk, Transfer learning enhanced physics informed neural network for phase-field modeling of fracture, Theor. Appl. Fract. Mech. 106 (2020) 102447.
[44] Somdatta Goswami, Minglang Yin, Yue Yu, George Em Karniadakis, A physics-informed variational DeepONet for predicting crack path in quasi-brittle materials, Comput. Methods Appl. Mech. Engrg. 391 (2022) 114587.
[45] Danial Amini, Ehsan Haghighat, Ruben Juanes, Physics-informed neural network solution of thermo-hydro-mechanical (THM) processes in porous media, 2022.
[46] Ehsan Haghighat, Ruben Juanes, SciANN: A Keras/TensorFlow wrapper for scientific computations and physics-informed deep learning using artificial neural networks, Comput. Methods Appl. Mech. Engrg. 373 (2021) 113552.
[47] Jürgen Schmidhuber, Deep learning in neural networks: An overview, Neural Netw. 61 (2015) 85–117.
[48] P. Sibi, S. Allwyn Jones, P. Siddarth, Analysis of different activation functions using back propagation neural networks, J. Theor. Appl. Inf. Technol. 47 (3) (2013) 1264–1268.
[49] Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, Jeffrey Mark Siskind, Automatic differentiation in machine learning: A survey, J. March. Learn. Res. 18 (2018) 1–43.
[50] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, Adam Lerer, Automatic differentiation in pytorch, in: 31st Conference on Neural Information Processing Systems, 2017.
[51] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al., {TensorFlow}: A system for {Large − Scale} machine learning, in: 12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 16, 2016, pp. 265–283.
[52] Diederik P. Kingma, Jimmy Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.
[53] Matthew D. Zeiler, Adadelta: An adaptive learning rate method, 2012, arXiv preprint arXiv:1212.5701.
[54] Xue Ying, An overview of overfitting and its solutions, J. Phys.: Conf. Ser. 1168 (2019) 022022.
[55] Kevin Linka, Amelie Schafer, Xuhui Meng, Zongren Zou, George Em Karniadakis, Ellen Kuhl, Bayesian physics-informed neural networks for real-world nonlinear dynamical systems, 2022.
[56] Hamid Reza Bayat, Shahed Rezaei, Tim Brepols, Stefanie Reese, Locking-free interface failure modeling by a cohesive discontinuous Galerkin method for matching and nonmatching meshes, Internat. J. Numer. Methods Engrg. 121 (8) (2020) 1762–1790.
[57] Jeremy Yu, Lu Lu, Xuhui Meng, George Em Karniadakis, Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems, Comput. Methods Appl. Mech. Engrg. 393 (2022) 114823.
[58] Xiaoxuan Zhang, Krishna Garikipati, Bayesian neural networks for weak solution of PDEs with uncertainty quantification, 2021.
[59] Zhenze Yang, Chi-Hua Yu, Markus J. Buehler, Deep learning model to predict complex stress and strain fields in hierarchical composites, Sci. Adv. 7 (15) (2021).
[60] Hengjie Wang, Robert Planas, Aparna Chandramowlishwaran, Ramin Bostanabad, Train once and use forever: Solving boundary value problems in unseen domains with pre-trained deep learning models, 2021, Preprint arXiv:2104.10873.